

BIT 4107 – Mobile Application Development

ENHANCED PROFESSIONAL STUDENT LOGBOOK

Student Name: CELESTINE GITAU

Admission Number: BIT/2024/54146

School/Faculty: COMPUTER AND INFORMATICS

Programme: MOBILE APPLICATION DEVELOPMENT

Academic Year: 2026

Semester: MAY/AUG

Lecturer: NYORO MICHAEL

Phone Number: 0769722238

Instructions

This logbook is used to document weekly learning activities, practical exercises, skills acquired, challenges encountered, solutions applied, and reflections. Students should complete all sections every week.

Each week to follow remarks in week 1 section.

Autogenerated Table of content

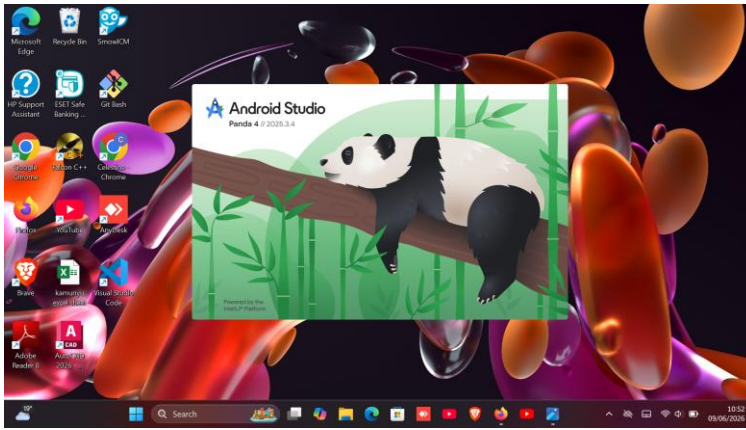
Week 1: Introduction to Mobile Application Development

Practical Activities Undertaken

1. Installed Android Studio and Java Development Kit (JDK).
2. Configured Android SDK and Gradle.
3. Created an Android Virtual Device (AVD) Emulator.
4. Created a new Android Studio project.
5. Explored Android Studio interface (Project Explorer, Code Editor, Layout Editor).
6. Executed the first Android application on the emulator.
7. Studied Android application architecture and project structure.
8. Tested successful project build and execution.

Screenshots to Attach

Android Studio Installation Completed



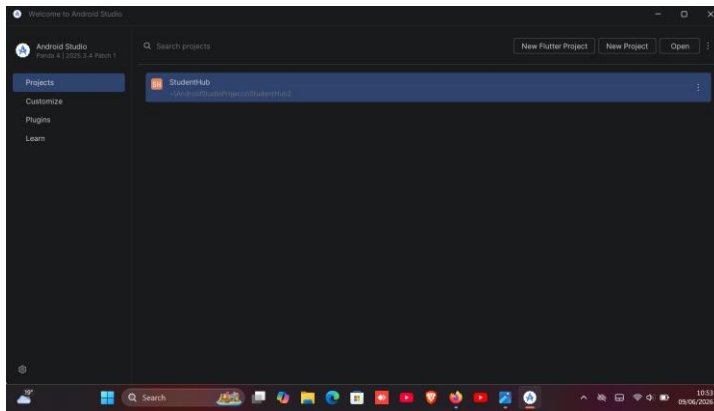


Figure 1.installation process

Created project structure

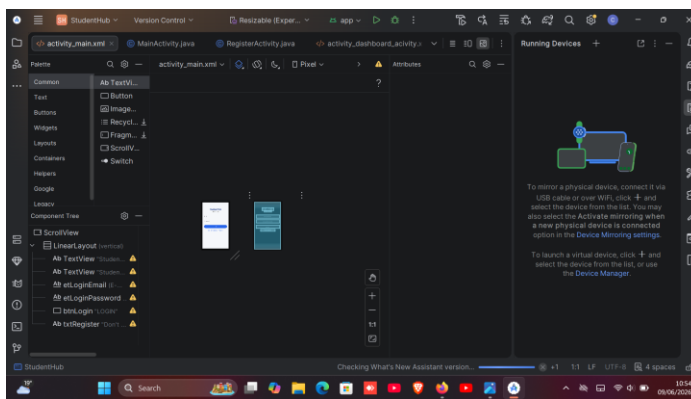
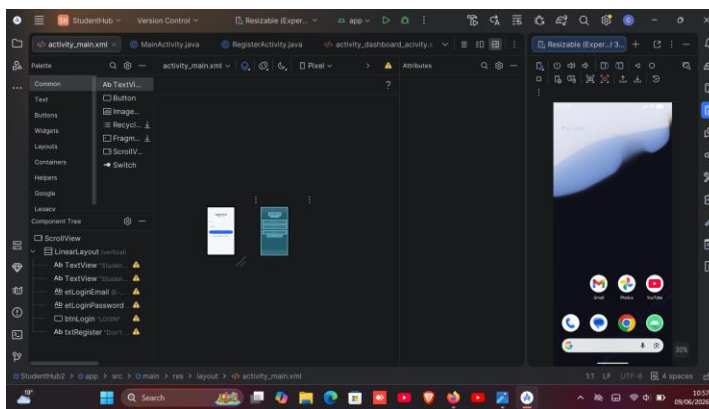


Figure 2.studenthub creation

Emulator working



Tools/Software Used

1. Android Studio
2. Java JDK
3. Android SDK
4. Android Emulator
5. Windows Operating System

Personal Reflection

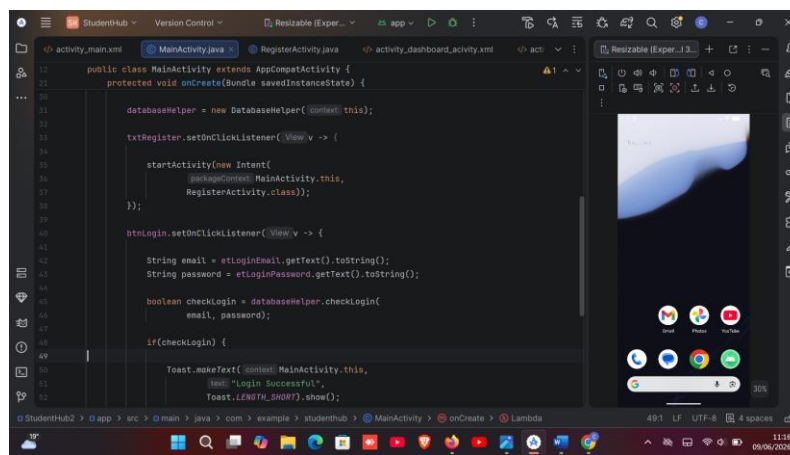
This week I successfully set up the Android development environment and understood the structure of Android projects. Running my first application on the emulator increased my confidence in mobile application development.

Practical Activities Undertaken

- ## Screenshots to Attach

The screenshot shows the Android Studio IDE. The main editor displays the `MainActivity.java` file, which is part of the `com.example.studenthub` package. The code defines a `MainActivity` class that extends `AppCompatActivity`. It includes imports for `EditText`, `Button`, `TextView`, and `DatabaseHelper`. The `onCreate` method is overridden, and it initializes the database helper, sets the content view to `R.layout.activity_main`, and finds the login email, login password, and login button views. The `onCreate` method is annotated with `@Override`.

On the right side, there is a preview of the app's UI, showing a login screen with a blue header, a white background, and a blue login button. The UI is displayed on a tablet device.



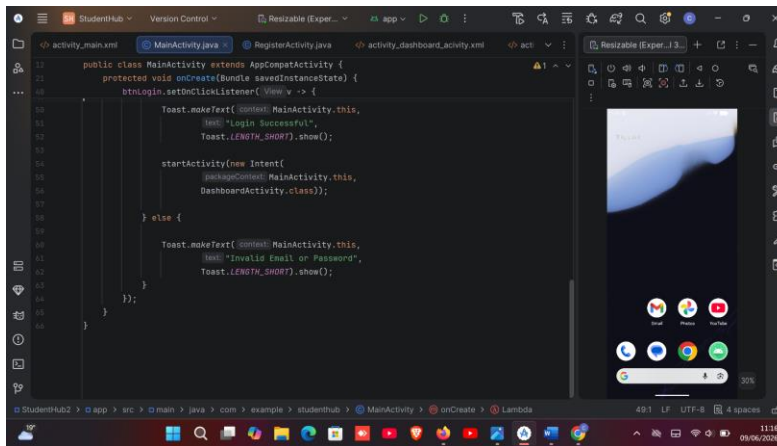
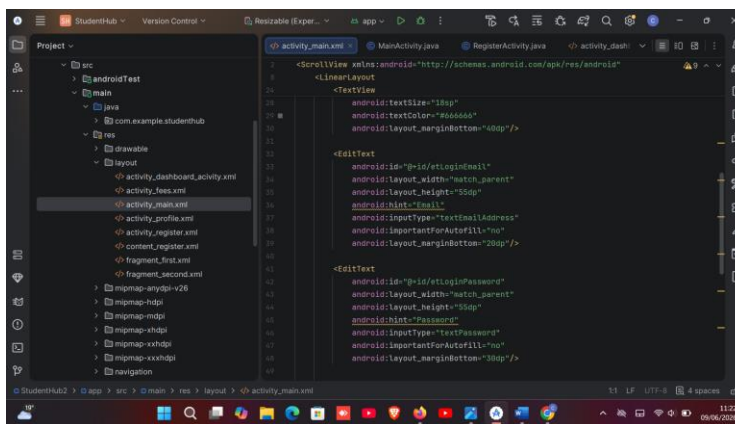
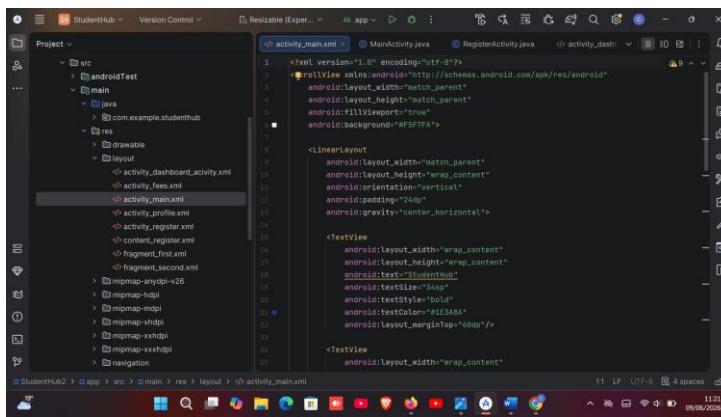


Figure 3.all are main activity.java

codes

Xml layout code



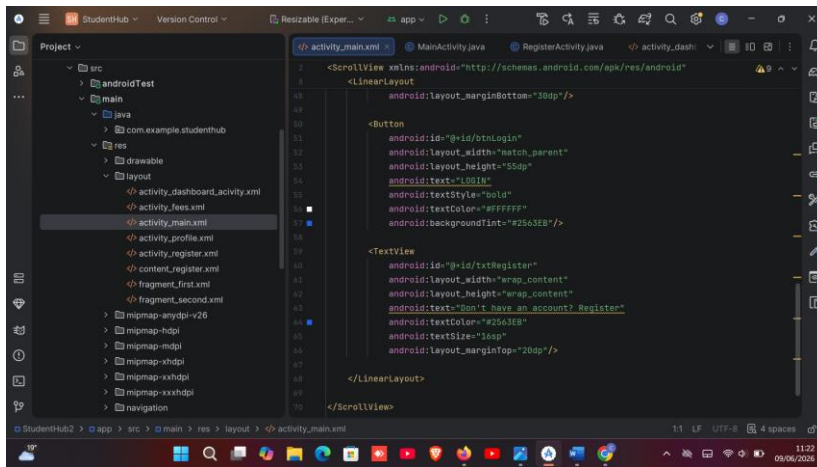


Figure 4.xml layout codes and

design

Login Screen Design

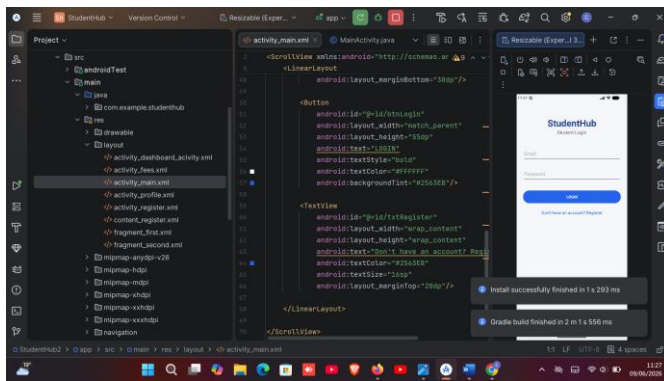


Figure 5login screen design

Student Registration Form

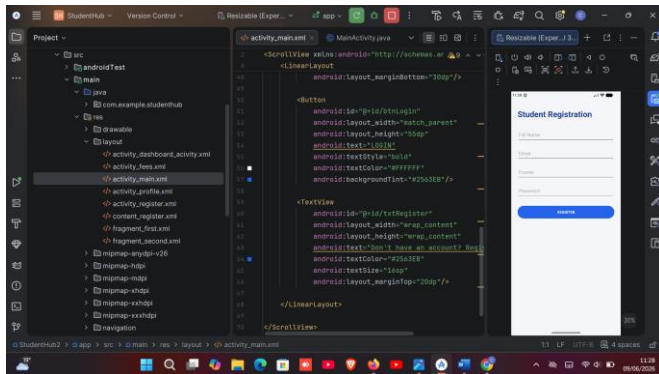


Figure 6.student registration form

Android Emulator Running Application

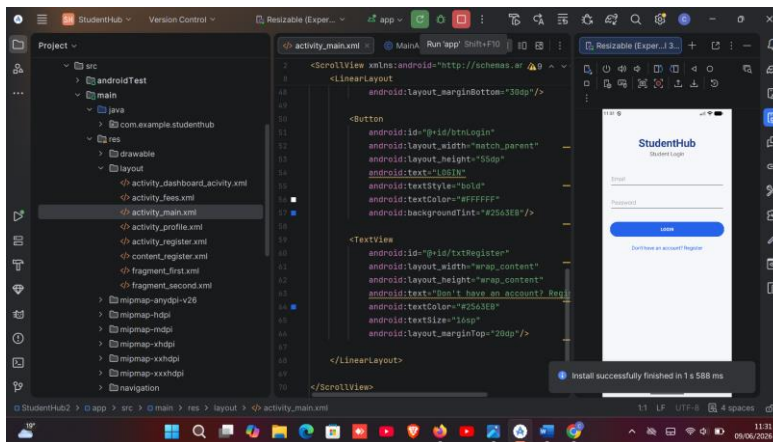
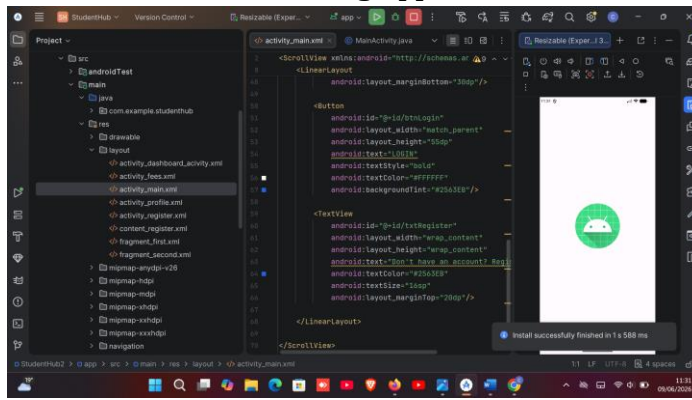


Figure 7 .app launching on emulator and running

Tools/Software Used

- Android Studio
- Java
- XML Layout Editor
- Android Emulator

Personal Reflection

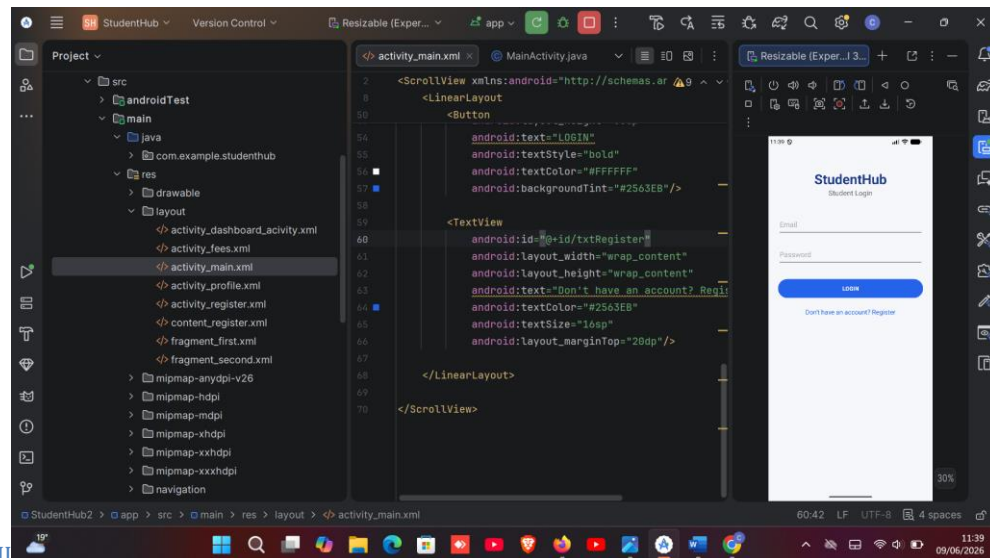
I improved my Java programming skills and learned how XML layouts work together with Java code. Designing the login and registration screens helped me understand Android user interface development.

Week 3: User Interface Design

Practical Activities Undertaken

1. Planned the Student Management App user interface.
2. Designed screen layouts using XML.
3. Created Login Screen.
4. Created Dashboard Screen.
5. Created Student Registration Form.
6. Implemented buttons and input fields.
7. Applied colors, spacing and alignment for a professional appearance.
8. Tested responsiveness of layouts on the Android emulator.
9. Improved navigation between screens.

Screenshots to Attach



Login Screen UI

Figure 8.login screen ui

Dashboard screen

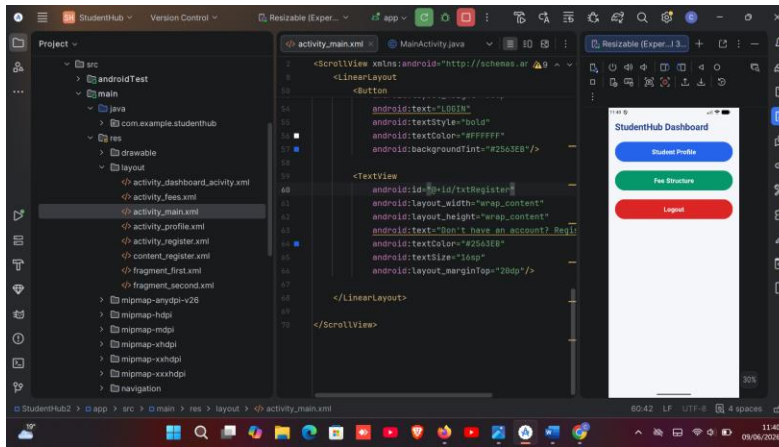


Figure 9.dashboard screen

Student registration screen

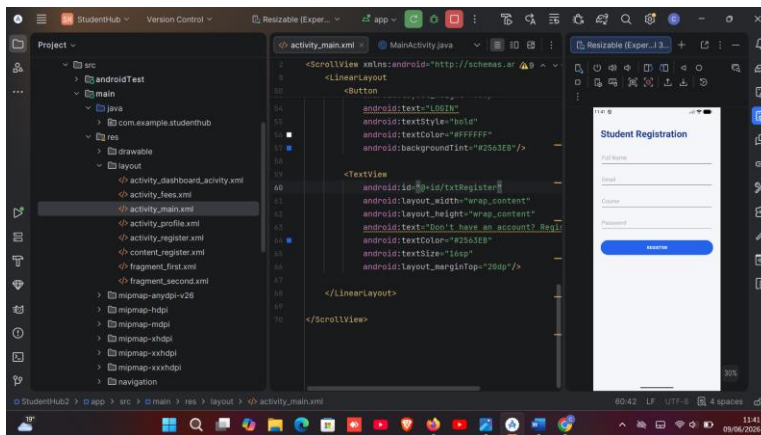
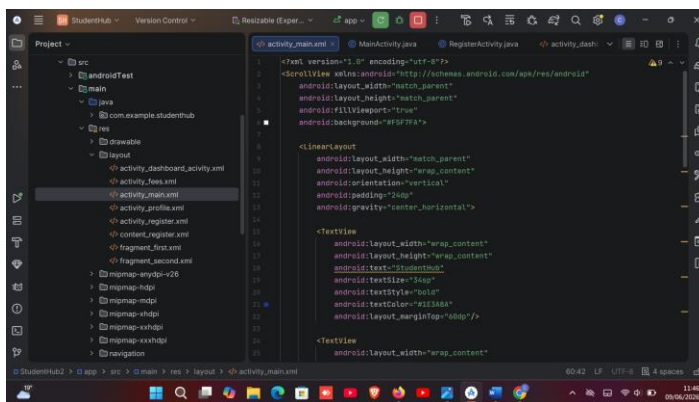


Figure 10.student registration

screen

Xml code editor



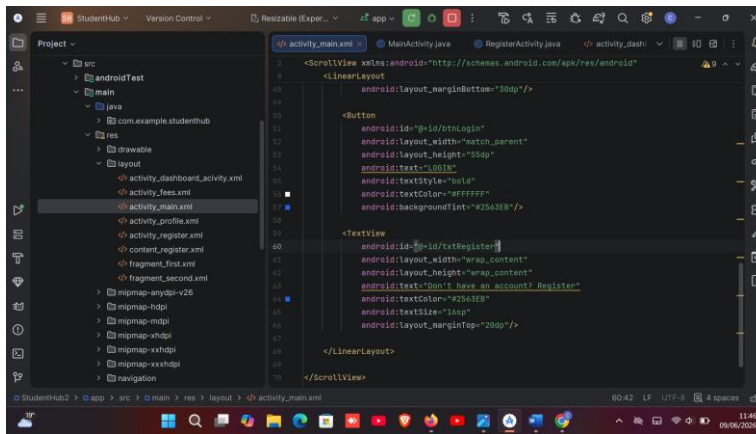
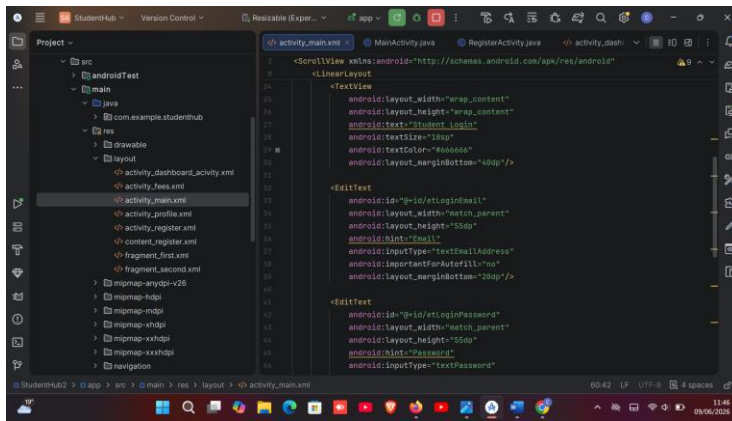


Figure 11.all xml editor codes

Navigation Between Screens

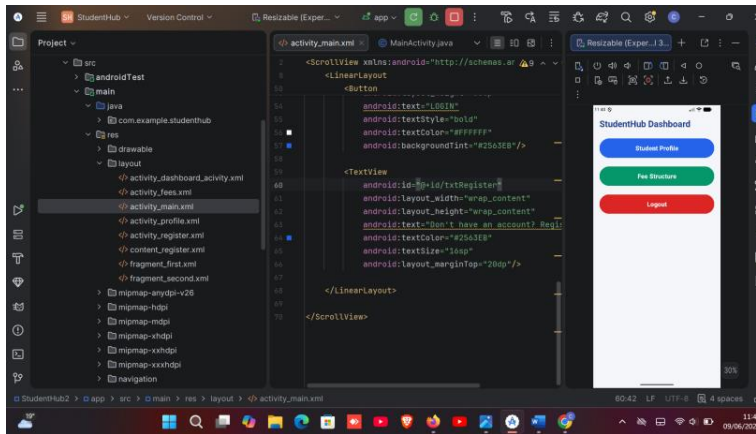


Figure 12.navigation btwn screens

Navigation to profile screens

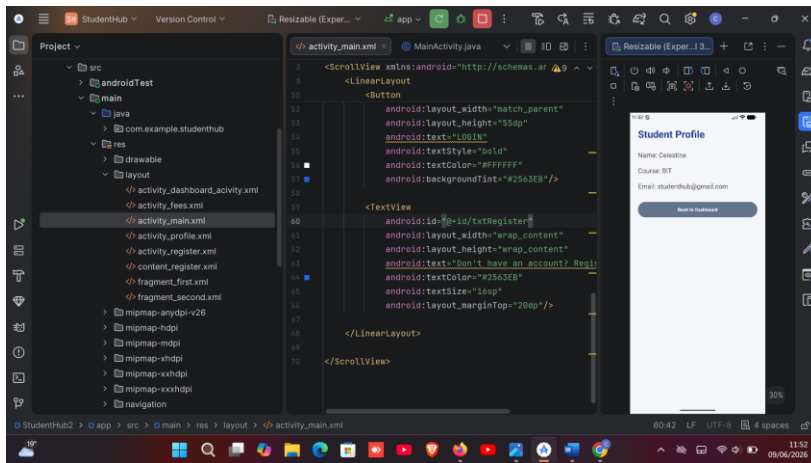


Figure 13.navigation to profile

screens

Navigation to fee structure screen

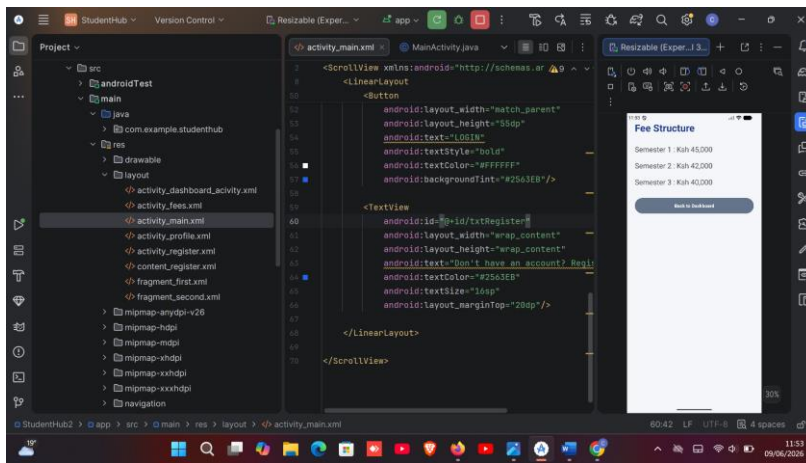
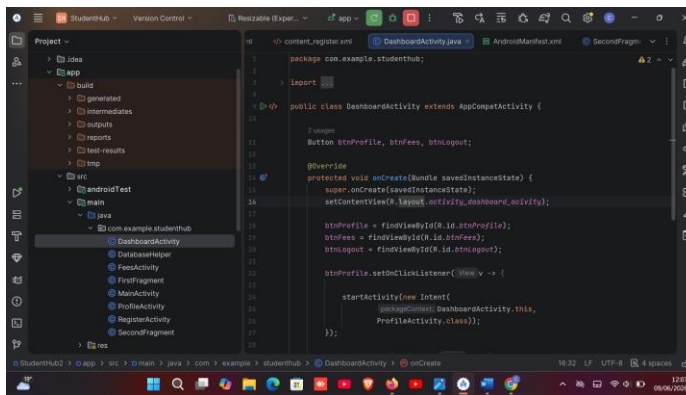


Figure 14.navigation to fee

structure

Java code implementing button clicks



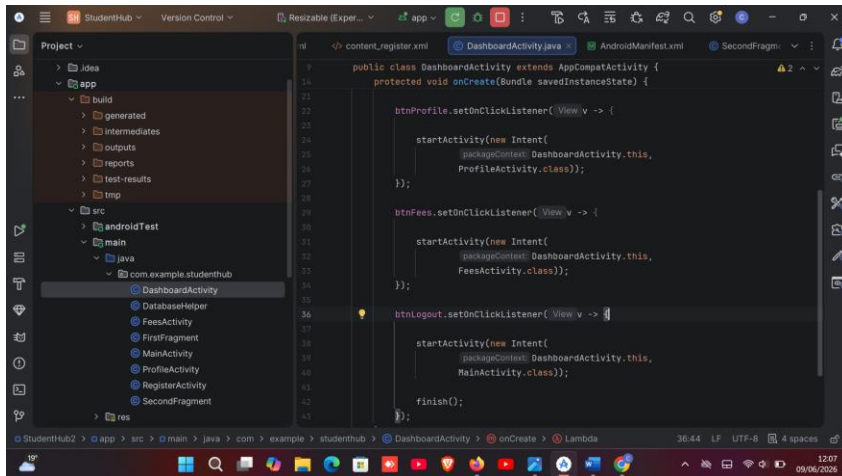


Figure 15.java code on

button clicks

Tools/Software Used

- Android Studio
- Java
- XML
- Android Emulator

Personal Reflection

This week I focused on user interface design and learned how to create attractive Android layouts using XML. The practical work improved my ability to design user-friendly screens and navigation within the application.

Week 4: Event Handling and User Interaction

Practical Activities Undertaken

1. Enhanced the Student Hub dashboard by replacing simple navigation buttons with professional Material Card Views.
2. Implemented click event handling using On Click Listener for all dashboard modules.
3. Added navigation between Profile, Course Registration, GPA Estimator, Timetable, E-Learning Center, Announcements, Student Records, Settings, and Logout screens.
4. Created smooth activity transitions between screens.
5. Implemented form validation for student registration and data entry fields.
6. Added button interactions for GPA calculation and E-Learning resources.
7. Improved user experience through organized dashboard layout and modern interface design.
8. Tested all user interactions using the Android Emulator.
9. Fixed navigation and activity launch errors.
10. Verified correct operation of all dashboard modules.

Android Studio Project Structure

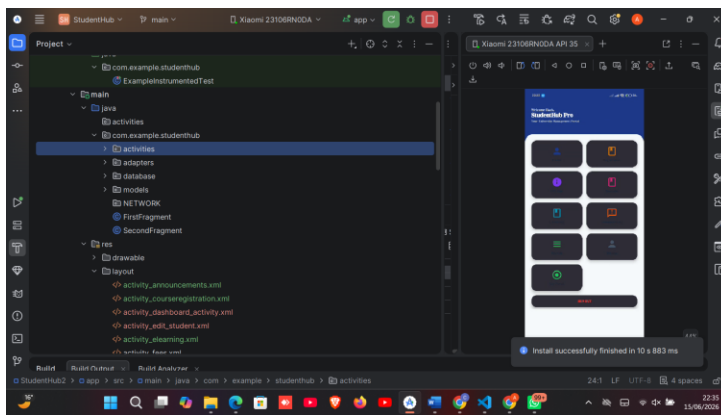
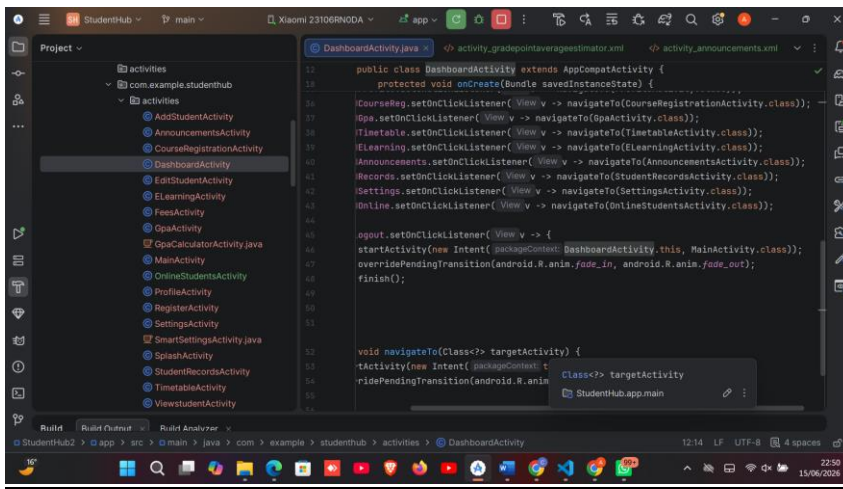
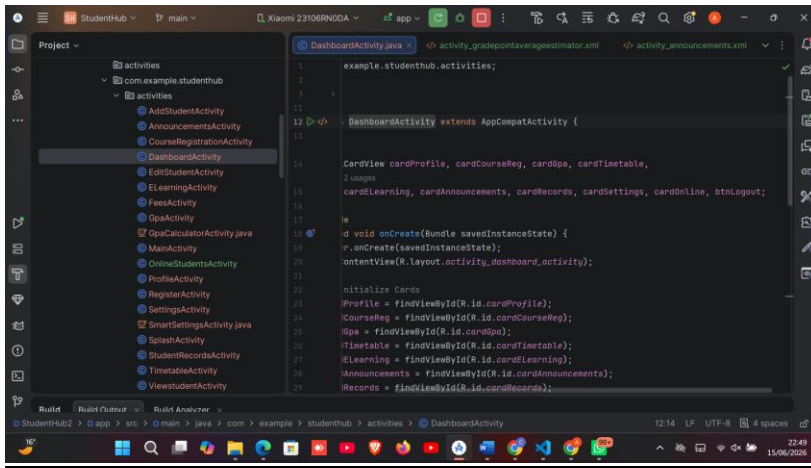


Figure 16 ANDROID STUDIO

STRUCTURE

DatabaseHelper.java



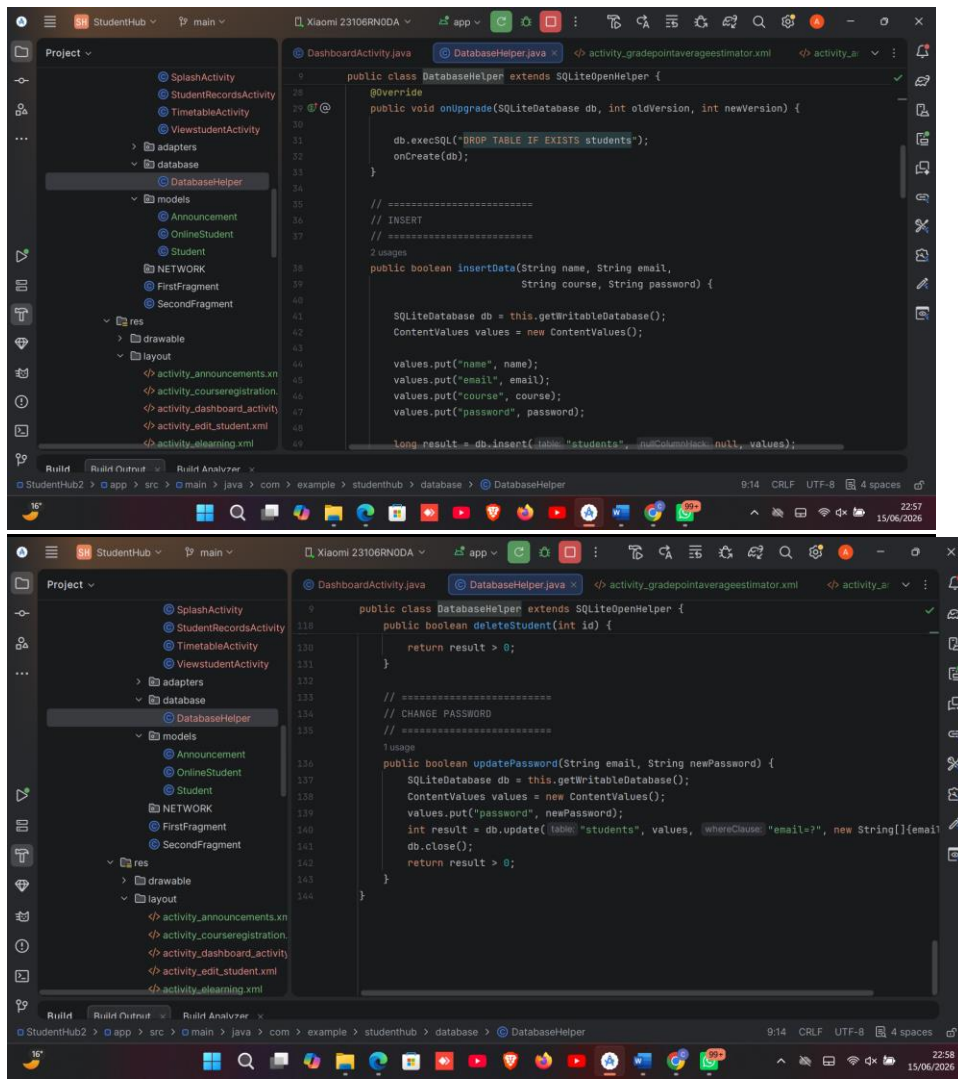


Figure 17 DATABASE_HELPER.JAVA CODES

Add Student Screen

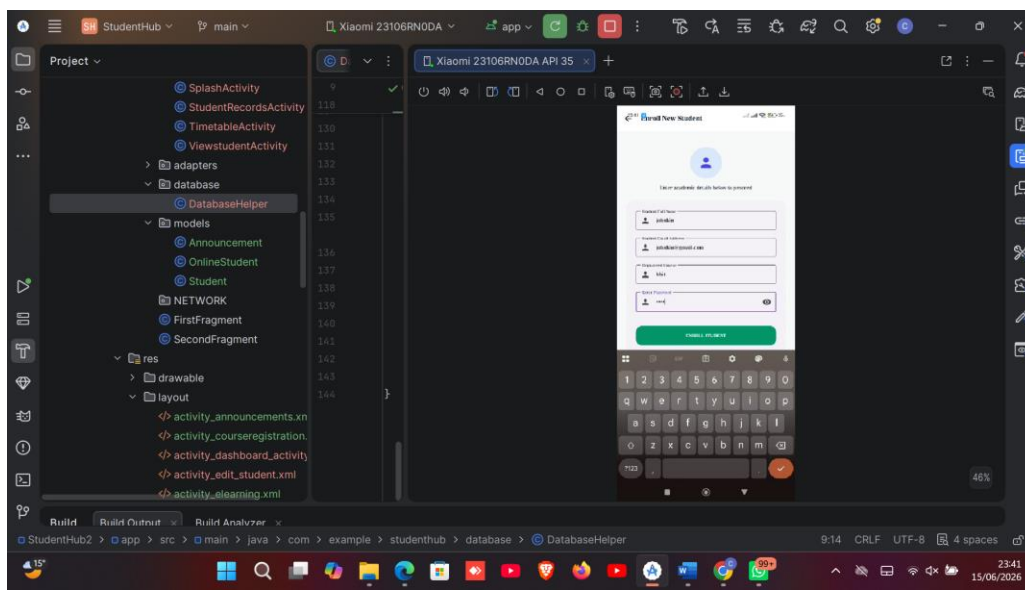
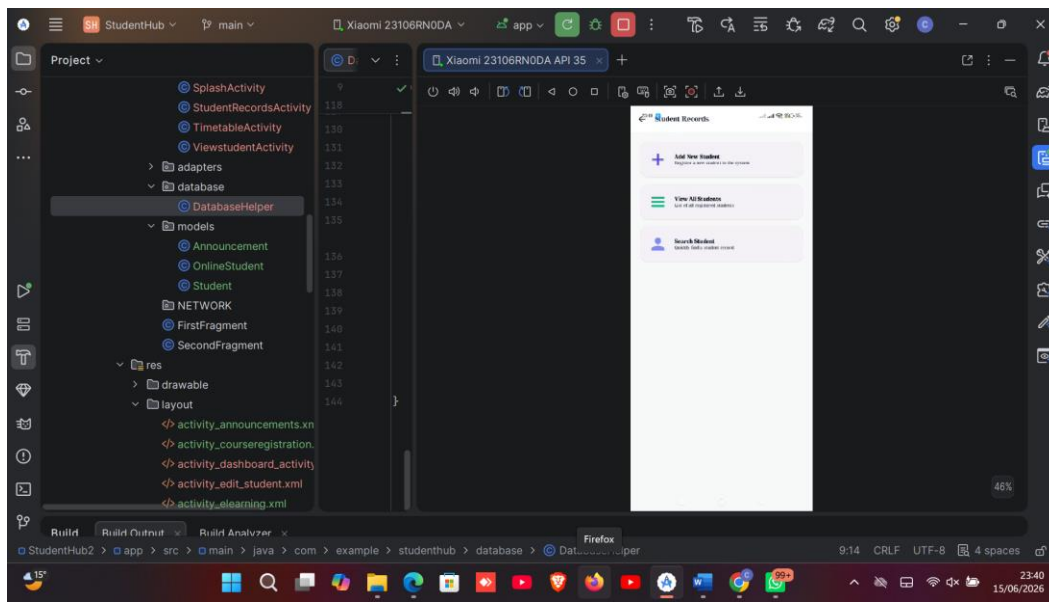


Figure 18ADD STUDENT

Student Saved Successfully

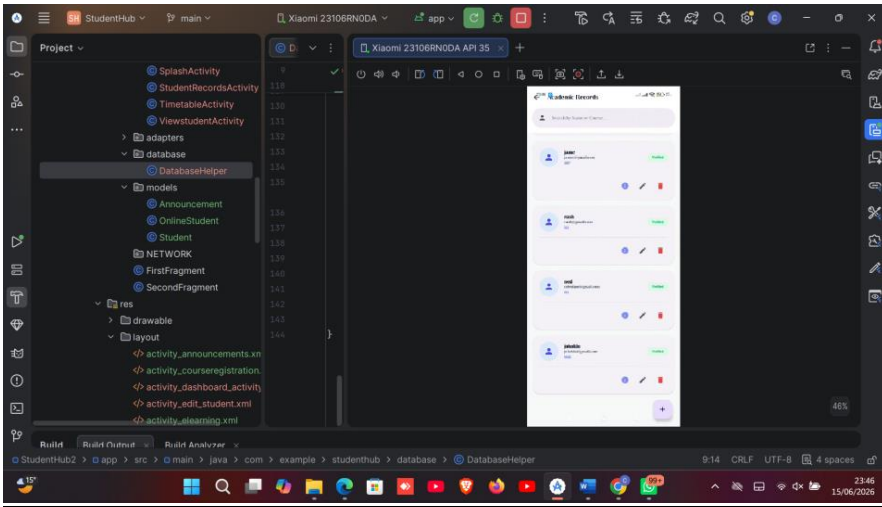


Figure 19STUDENT SAVED

View Students Screen

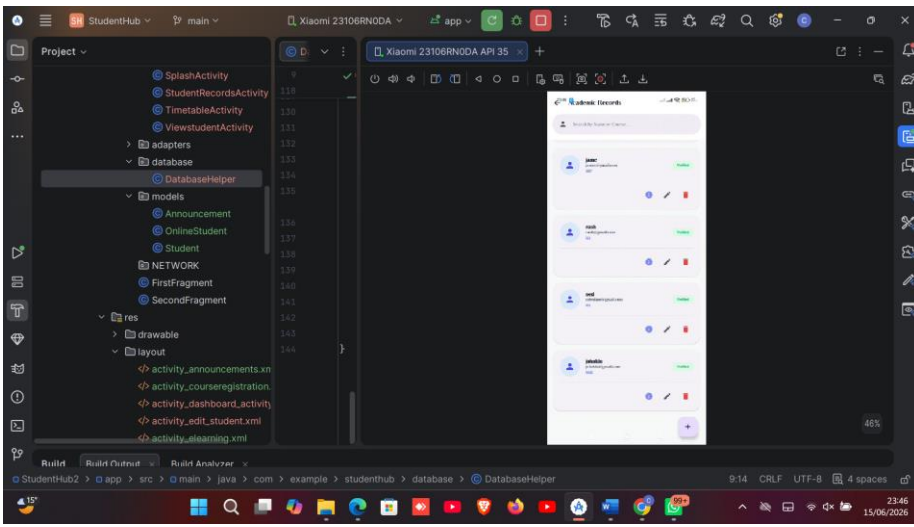
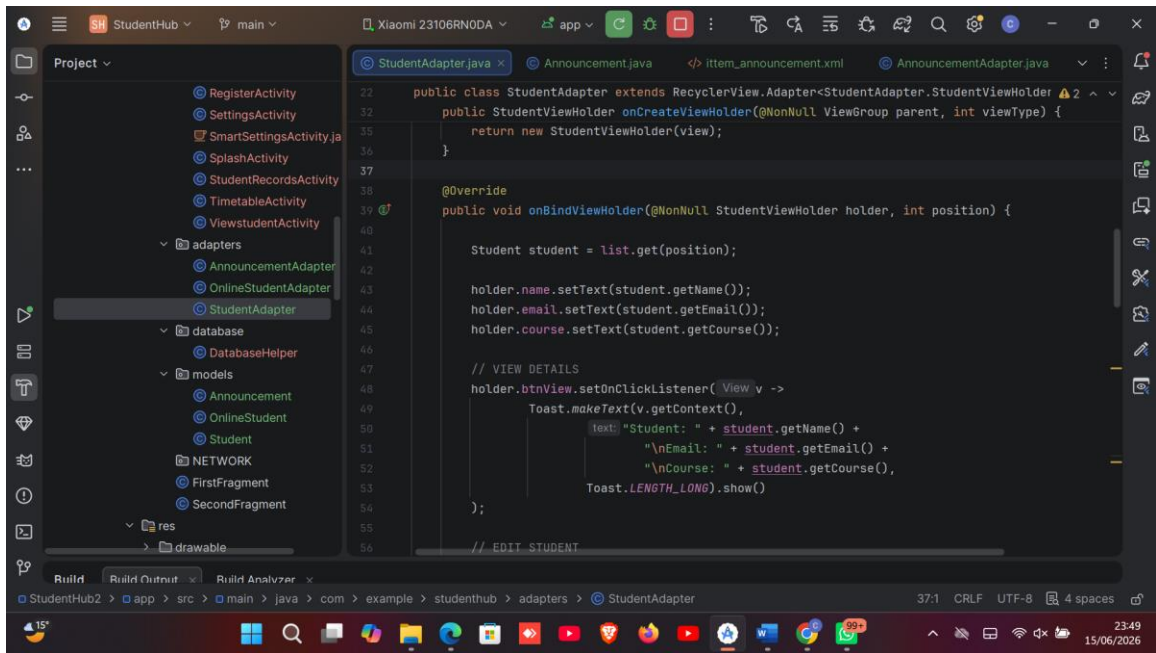
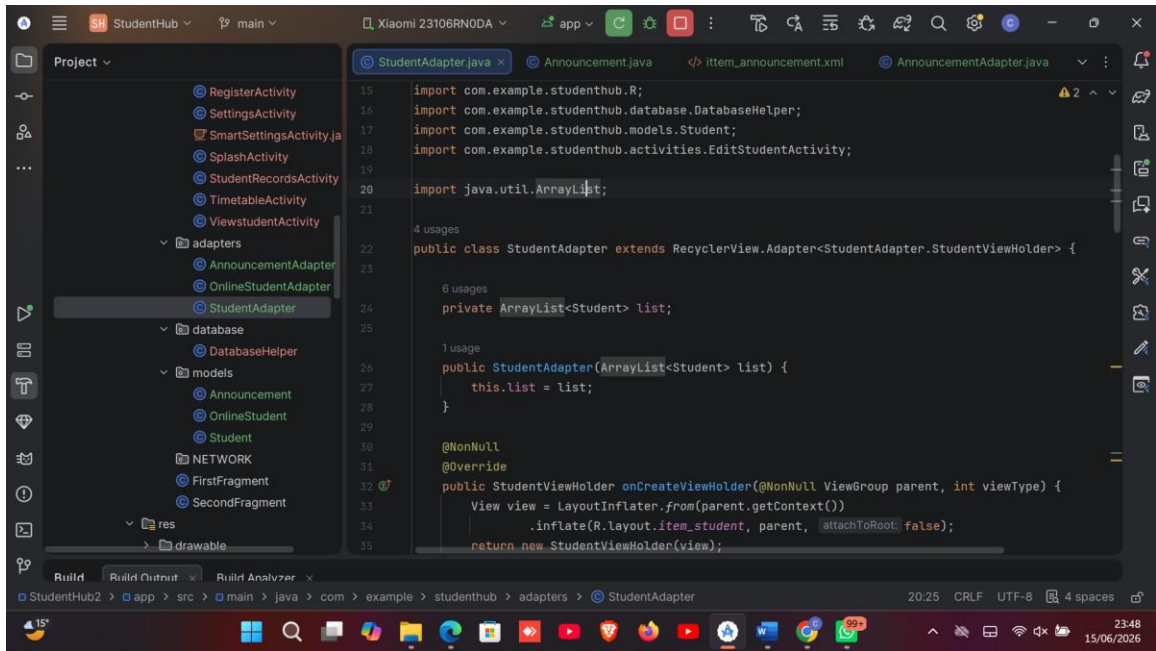
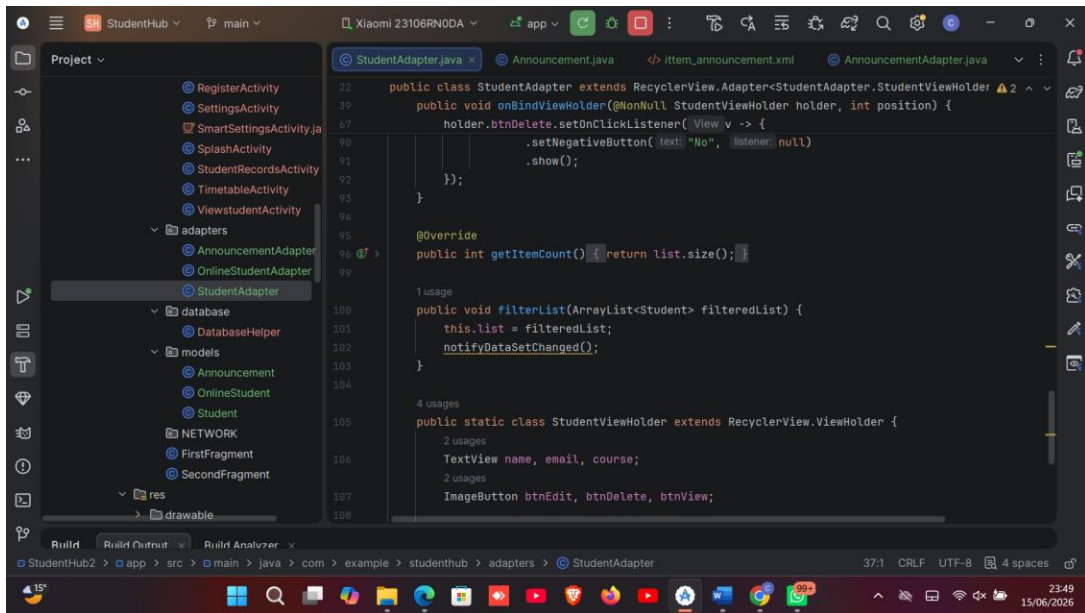
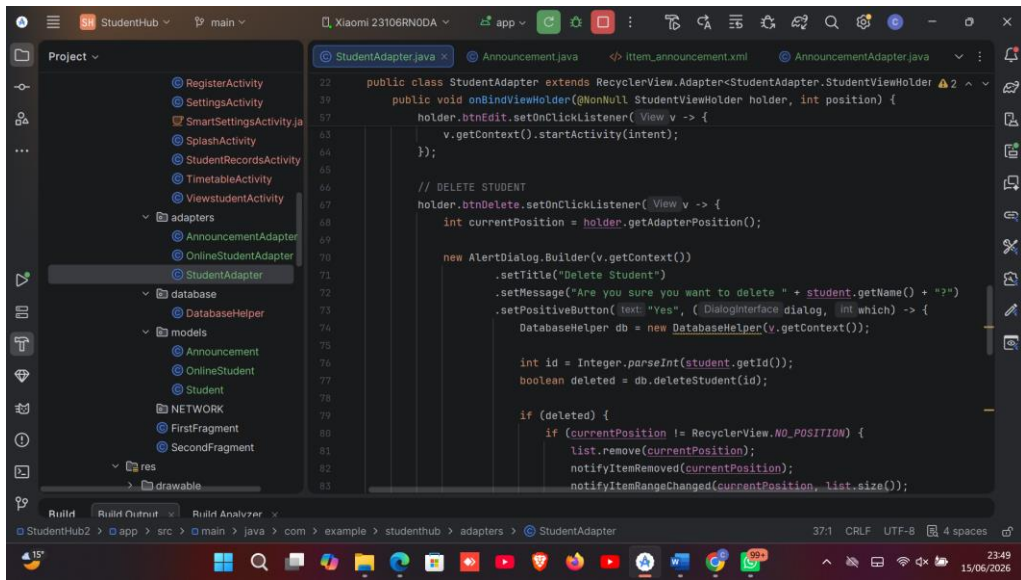


Figure 20Figure 21. RecyclerView displaying stored student records.

RecyclerView Adapter





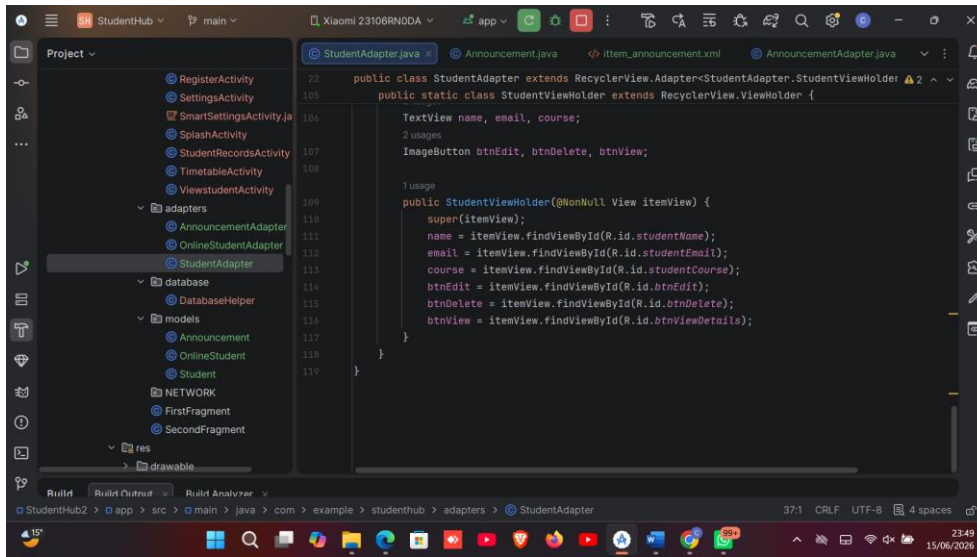


Figure 21Figure 22. RecyclerView adapter implementation.

Update Student

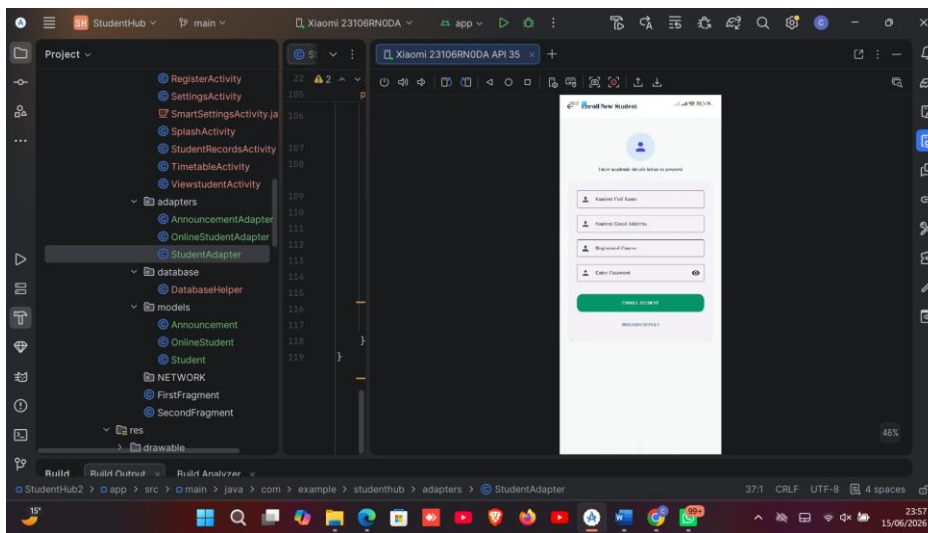
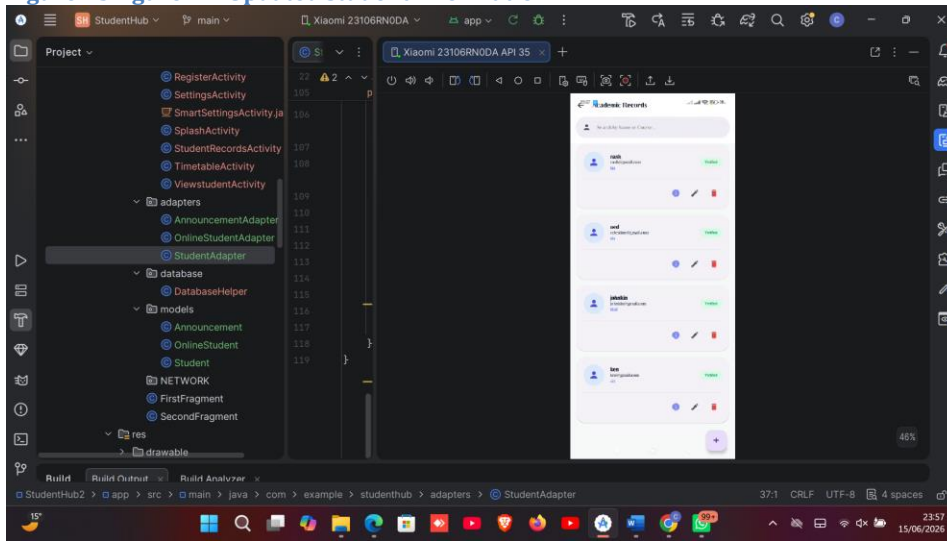


Figure 22Figure 23.

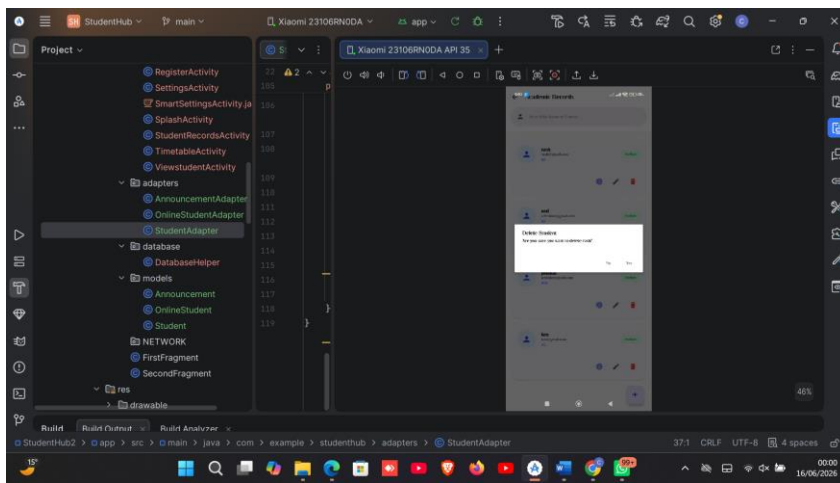
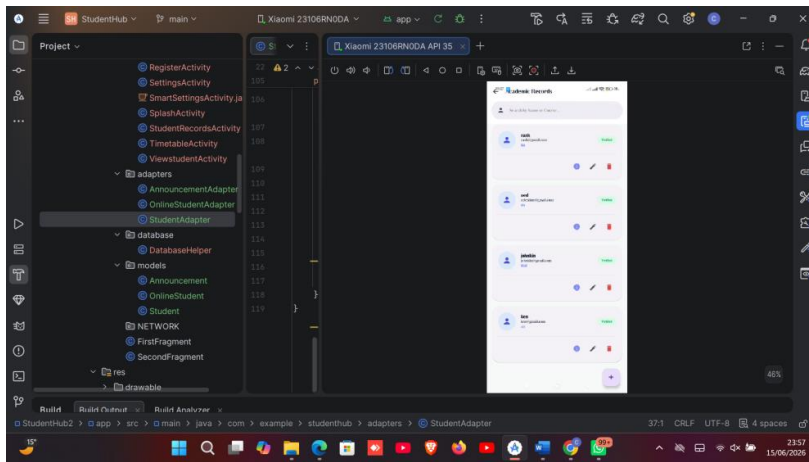
Student record update screen.

Updated record

Figure 23Figure 24. Updated student information.



Delete Function



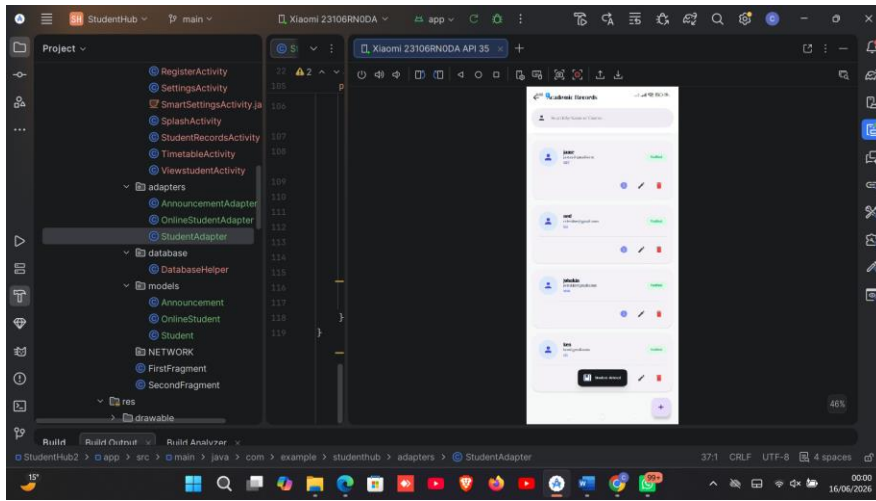
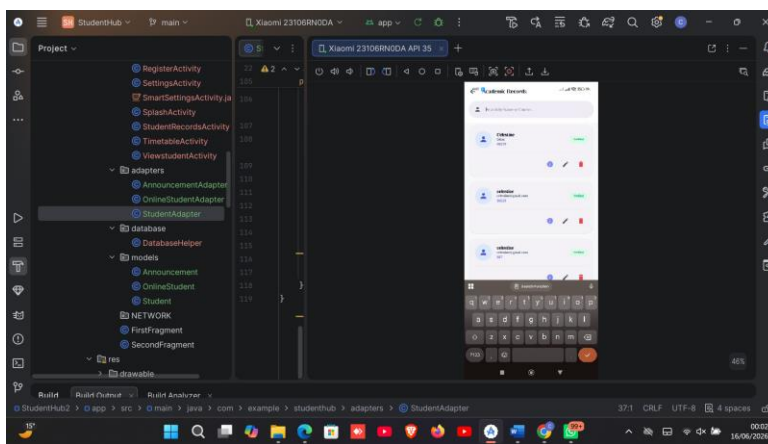
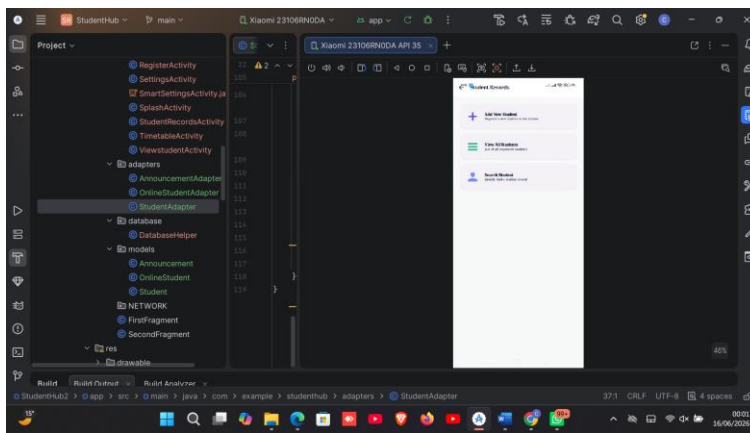


Figure 24Figure 25. Delete student functionality.

Search Function



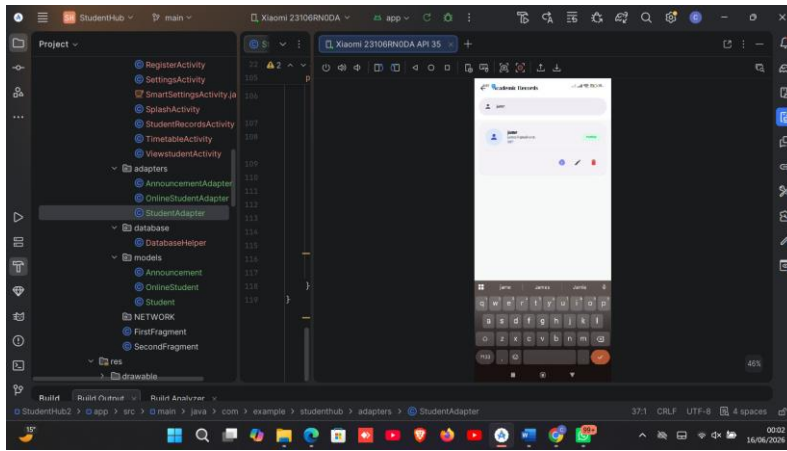
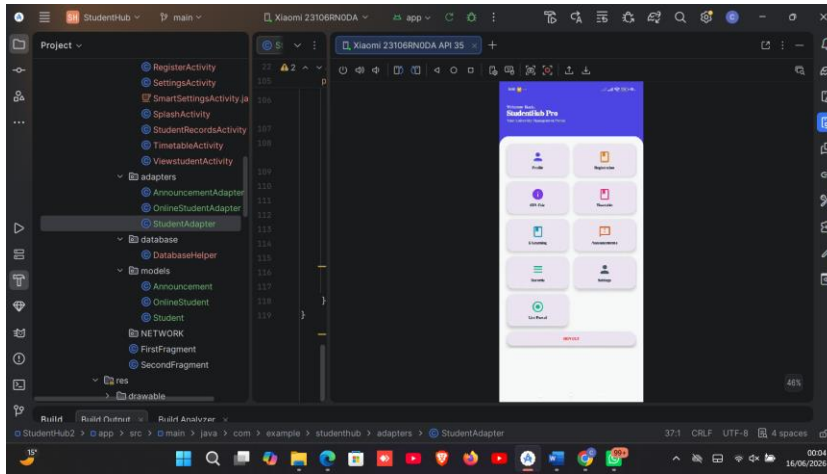


Figure 25 Figure 26. Student search functionality.

Dashboard With New Modules



Tools/Software Used

- Android Studio
- Java Programming Language
- XML Layout Editor
- SQLite Database
- RecyclerView
- Android Emulator
- Material Design Components

PRACTICAL ACTIVITIES

Designed and implemented SQLite database storage for Student Hub. Created a students table and implemented CRUD operations including adding, viewing, updating, deleting, and searching student records. Developed a RecyclerView-based records screen to display stored information efficiently. Expanded the Student Hub dashboard by adding Student Records, GPA Estimator, Timetable, E-Learning, and Settings modules. Tested all database operations and verified successful storage and retrieval of student information.

Week 5: Data Management

Practical Activities Undertaken

1. Designed and implemented an SQLite database for the Student Hub application.
2. Created a DatabaseHelper class to manage database operations.
3. Developed a students table for storing student records.
4. Implemented student registration and data storage functionality.

5. Implemented CRUD operations (Create, Read, Update, Delete).
6. Developed a RecyclerView to display student records.
7. Added student search functionality for easy record retrieval.
8. Integrated the Student Records java module with the dashboard.
9. Tested database storage and retrieval processes.
10. Verified successful updating and deletion of student records.

Announcements Module

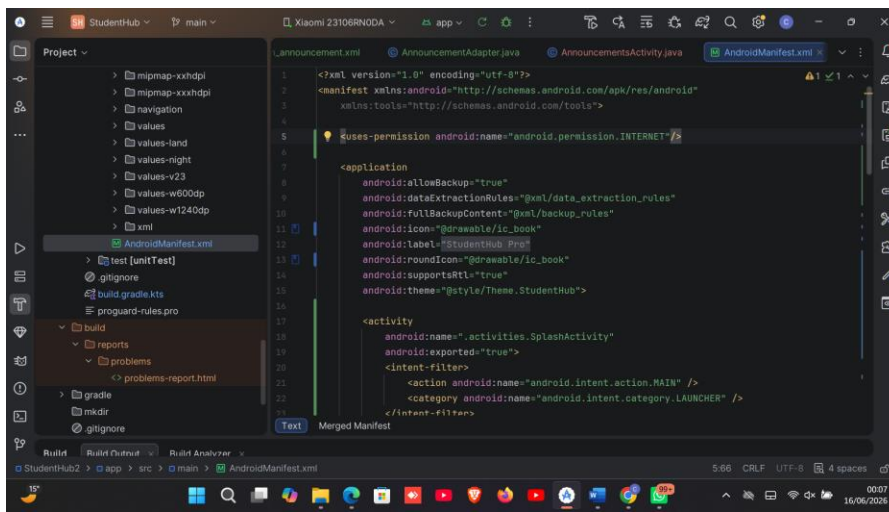
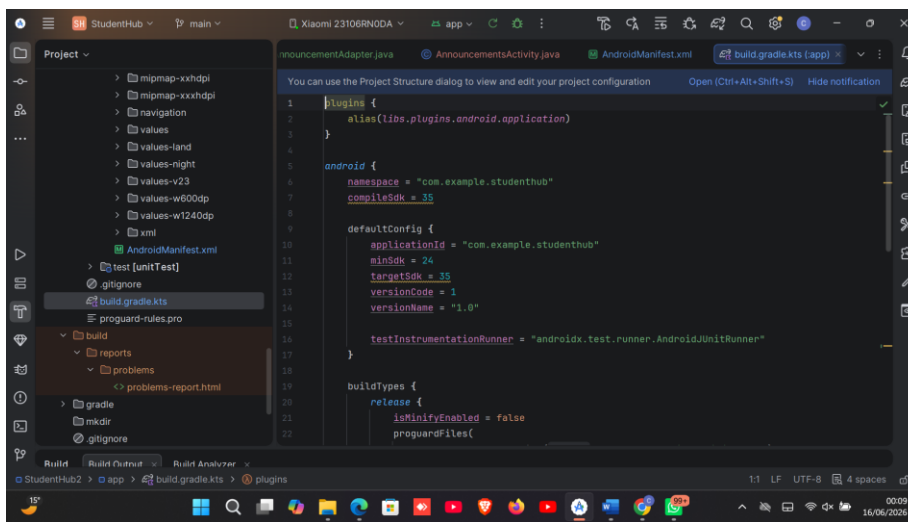


Figure 26ANNOUNCEMENT MODULE

build.gradle



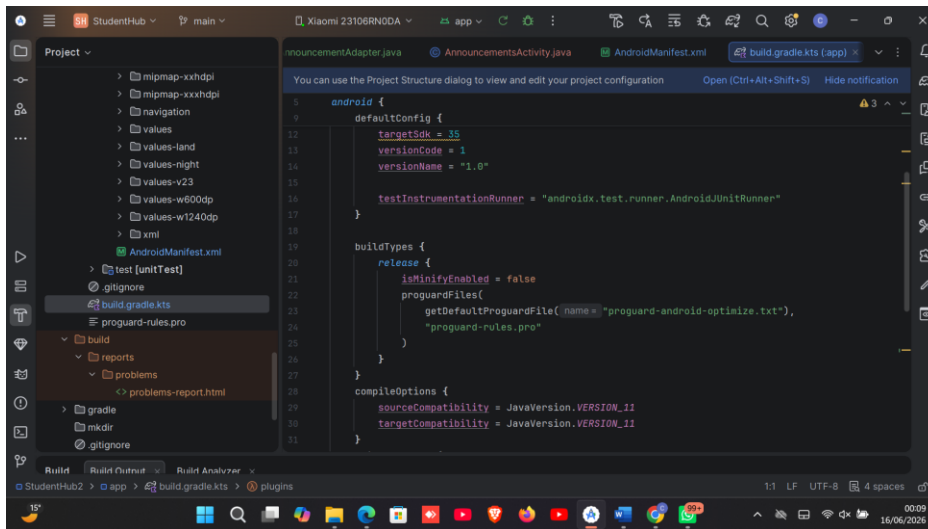
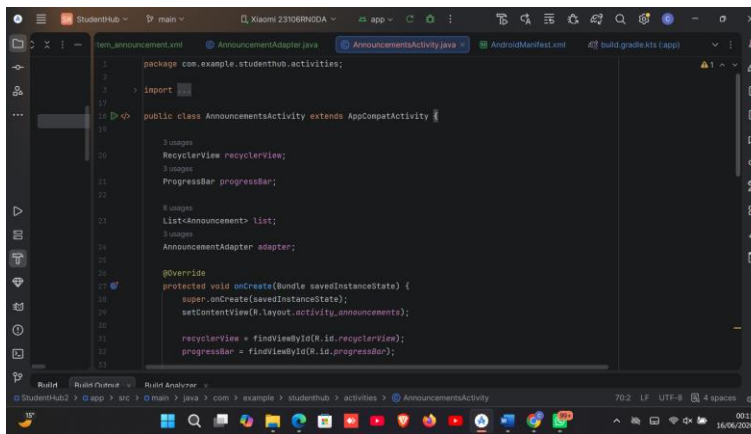


Figure 27 BUILD GRADLE KTS

Announcement.java code



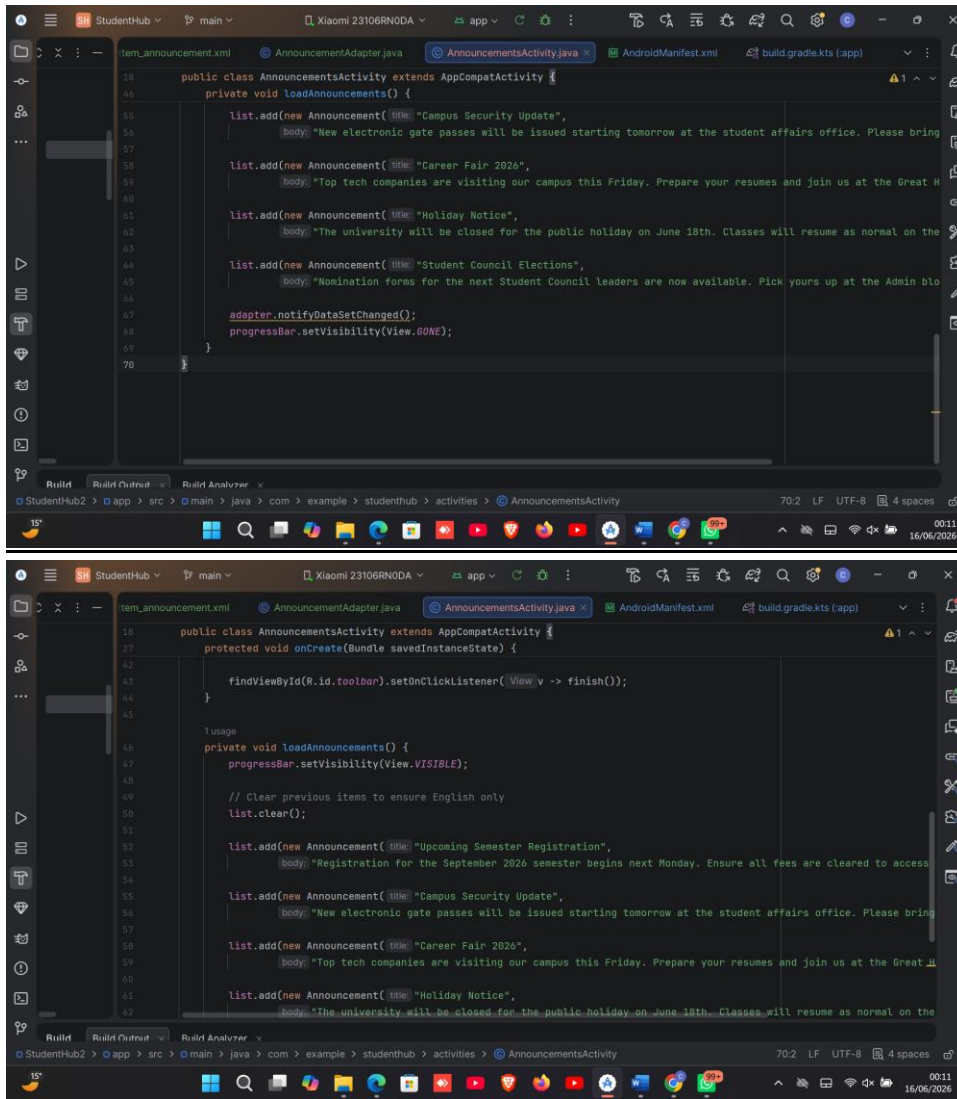
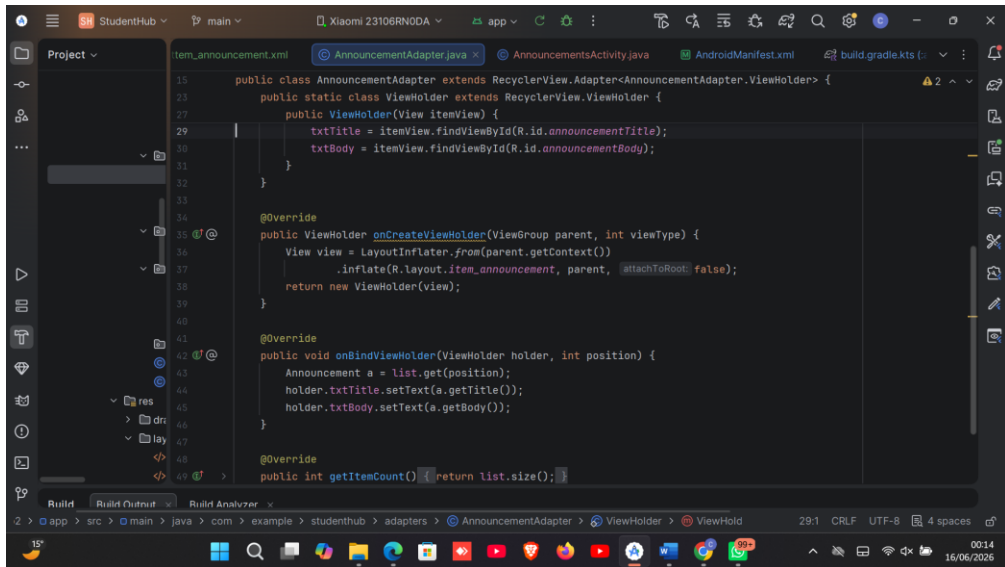
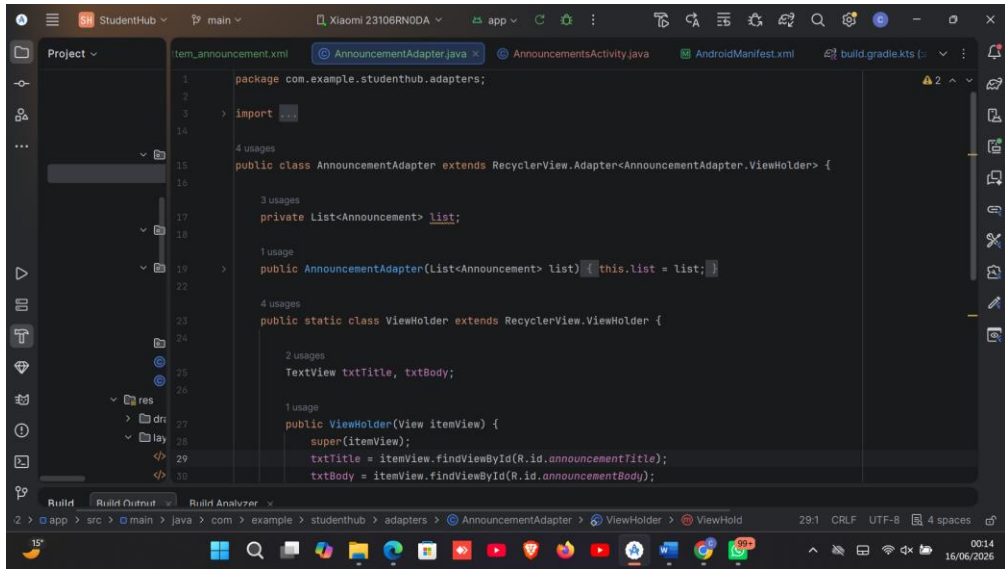


Figure 28ANNOUNCEMENT JAVA CODE

AnnouncementAdapter.java



Figure

29ANNOUNCEMENT ADAPTER.JAVA

Show loaded announcements.

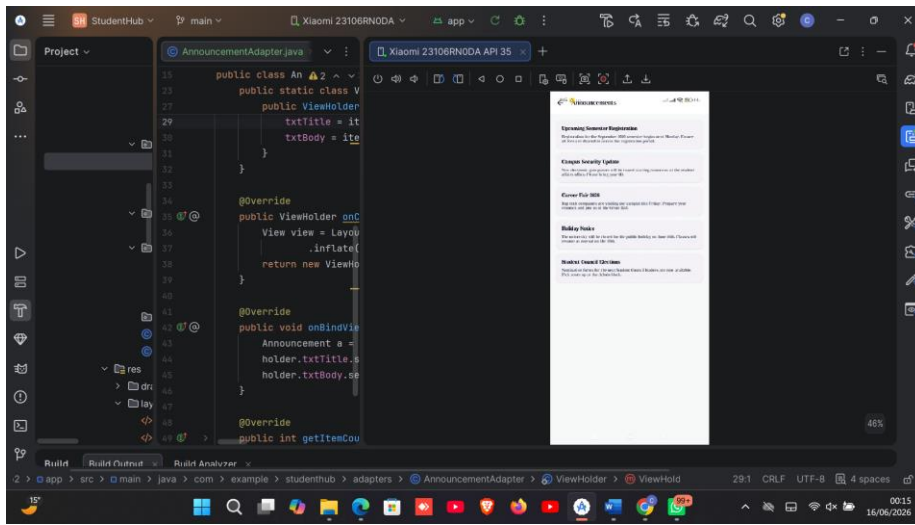


Figure 30ANNOUNCEMENT SCREEN

Live portal Figure JSON data parsing.

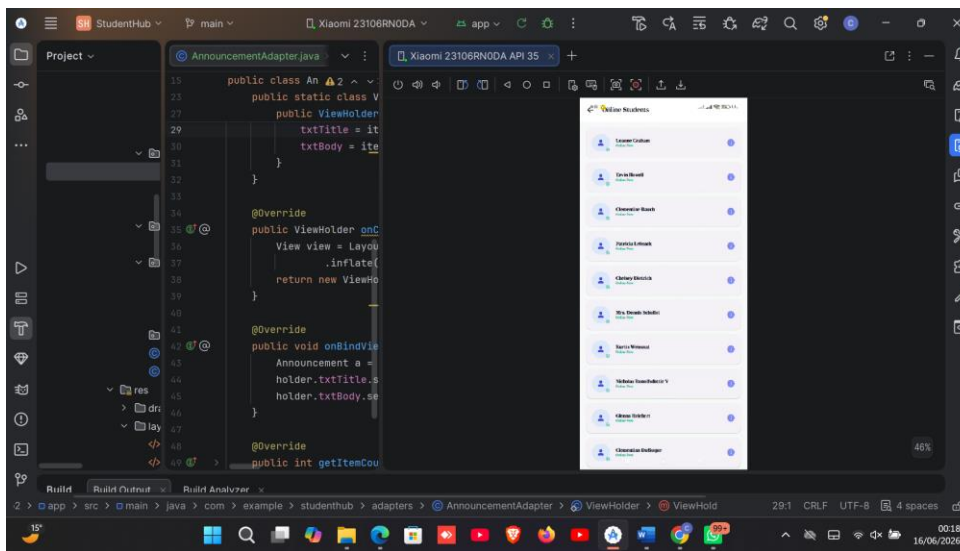


Figure 31LIVE PORTAL SCREEN

Tools/Software Used

- Android Studio
- Java Programming Language
- XML Layout Editor
- Volley Networking Library
- REST API (JSONPlaceholder API)

- JSON Data Format
- RecyclerView
- Android Emulator
- Internet Connectivity Tools (HTTP Requests)

PERSONAL REFLECTION

I gained practical experience in database management using SQLite. Implementing CRUD operations and RecyclerView improved my understanding of data storage, retrieval, and presentation in Android applications. This enhanced the functionality and reliability of the Student Hub project.

Week 6: Networking and APIs

Practical Activities Undertaken

1. Studied networking concepts and APIs.
2. Configured internet permissions in the application.
3. Added the Volley networking library.
4. Created models and RecyclerView adapters for online data.
5. Implemented HTTP requests and consumed a REST API.
6. Parsed JSON responses and displayed data in RecyclerView.
7. Implemented network error handling.
8. Integrated API features into the Announcements and Live Portal modules.
9. Tested API connectivity and data retrieval.

Show project structure:

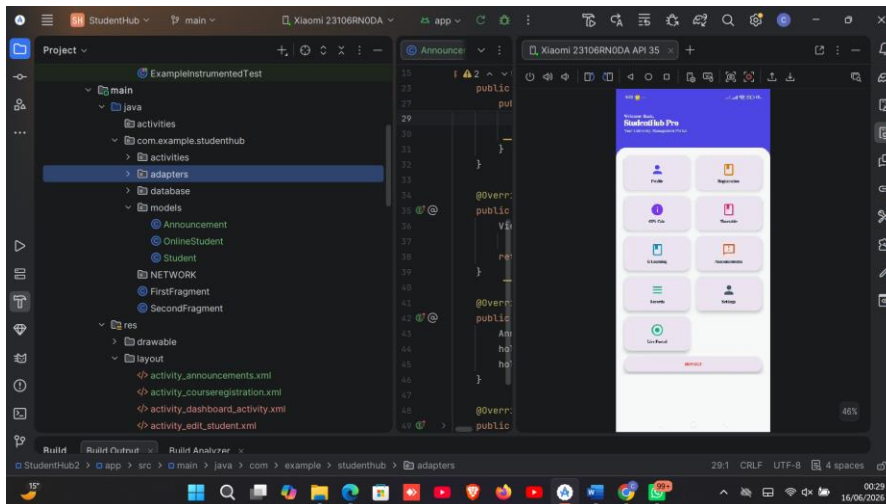


Figure 32PROJECT STRUCTURE

API MODEL

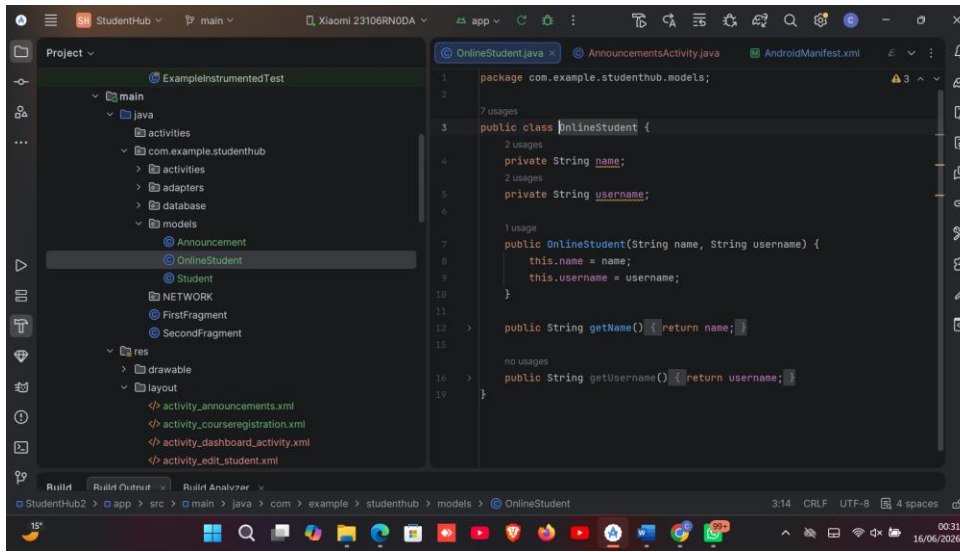
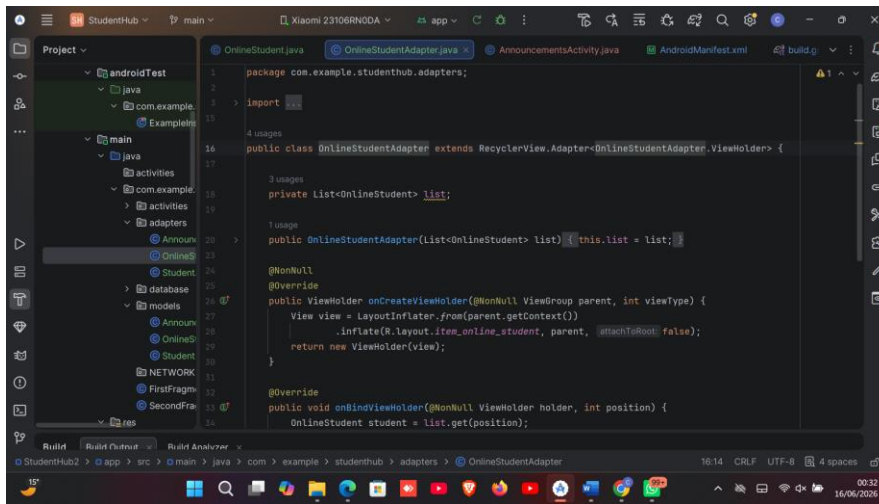


Figure 33 API MODEL

API ADAPTER



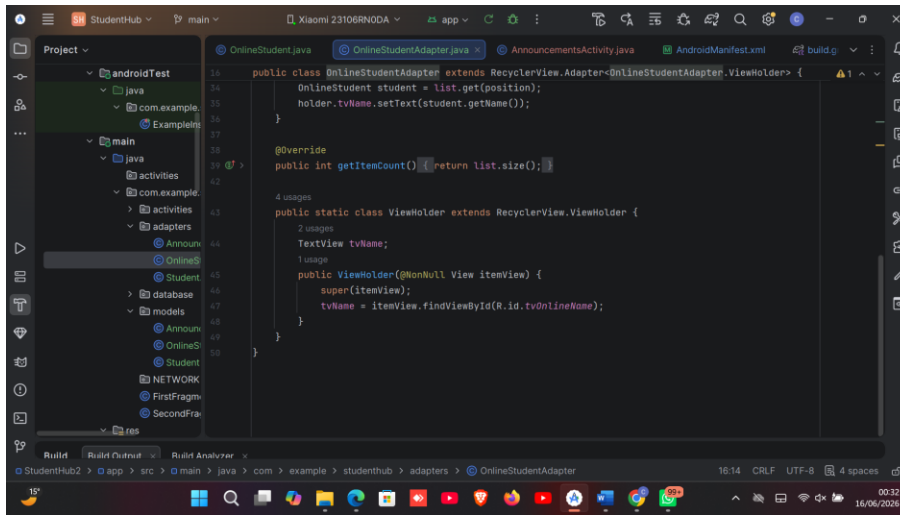
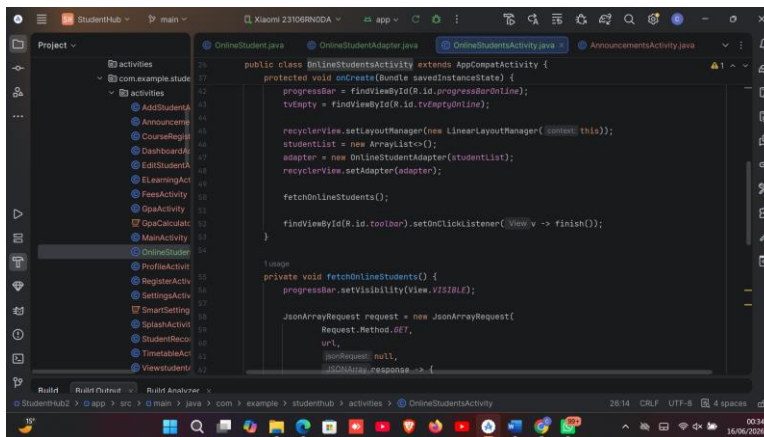
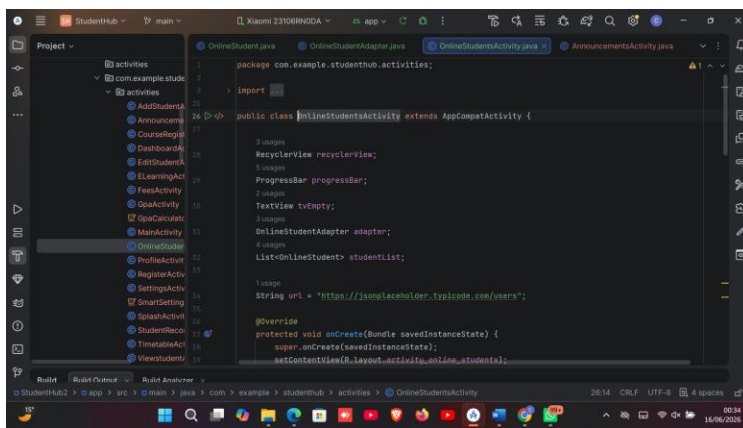


Figure 34 API ADAPTER

API REQUEST CODE



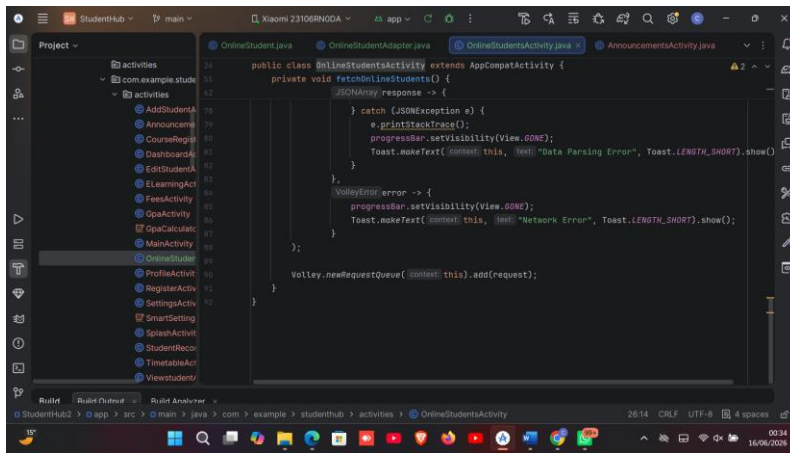


Figure 35Figure 41. HTTP GET request implementation using Volley.

RUN API SCREEN

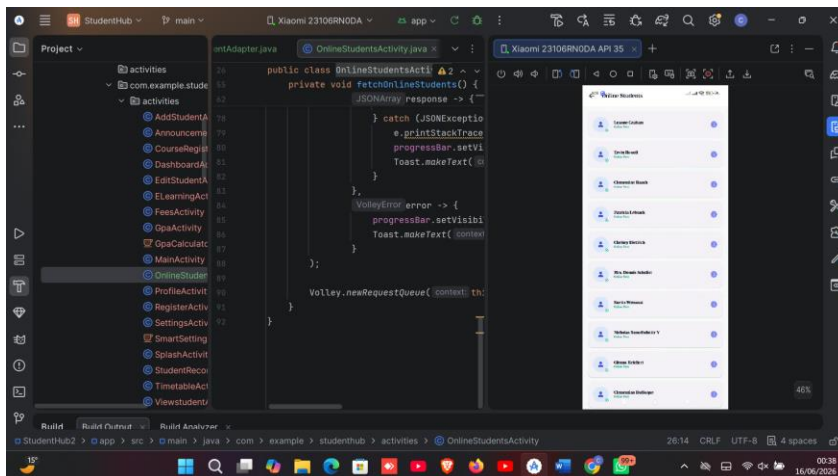


Figure 36Figure 43. API records successfully loaded from the internet.

SHOW RECYCLERVIEW

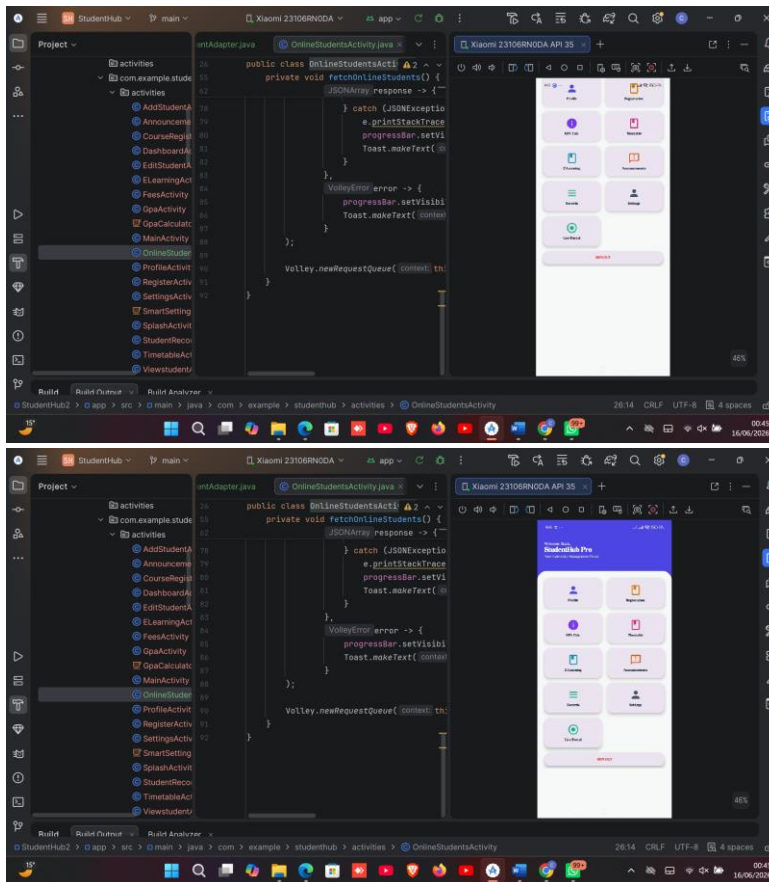


Figure 37Figure 45. RecyclerView displaying records retrieved from API.

TOOLS USED

- Android Studio
- Java
- XML
- Volley Networking Library
- REST API
- JSON
- RecyclerView
- Android Emulator
- Internet Connectivity

PERSONAL REFLECTION

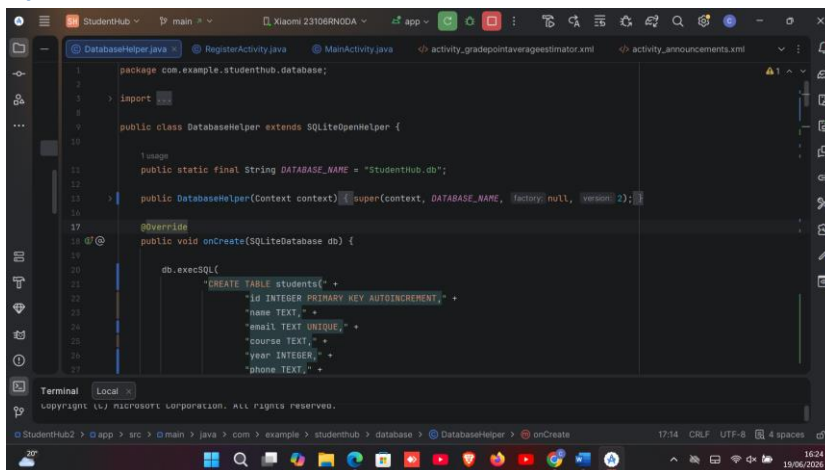
This week enhanced my understanding of mobile networking and API integration. I learned how to retrieve online data, process JSON responses, and display information dynamically. Implementing error handling improved the robustness of the Student Hub application and prepared me for real-world mobile development.

Week 7: Application Architecture

PRACTICAL ACTIVITIES

- Enhanced SQLite database structure
- Added Attendance table with relational design
- Implemented password hashing for secure storage
- Used SharedPreferences for session persistence
- Developed Attendance recording module
- Built Attendance Reports using RecyclerView
- Retrieved and displayed profile data from SQLite
- Tested full data storage and retrieval workflow

Updated Database Structure



```
1 package com.example.studenthub.database;
2
3 import androidx.sqlite.db.SupportSQLiteOpenHelper;
4
5 public class DatabaseHelper extends SQLiteOpenHelper {
6
7     //usage
8     public static final String DATABASE_NAME = "StudentHub.db";
9
10    public DatabaseHelper(Context context) {
11        super(context, DATABASE_NAME, null, 2);
12    }
13
14    @Override
15    public void onCreate(SQLiteDatabase db) {
16
17        db.execSQL(
18            "CREATE TABLE students( +
19                \"id INTEGER PRIMARY KEY AUTOINCREMENT,\" +
20                \"name TEXT,\" +
21                \"email TEXT UNIQUE,\" +
22                \"course INTEGER,\" +
23                \"year INTEGER,\" +
24                \"phone TEXT,\" +
25            );
26        );
27    }
28 }
```

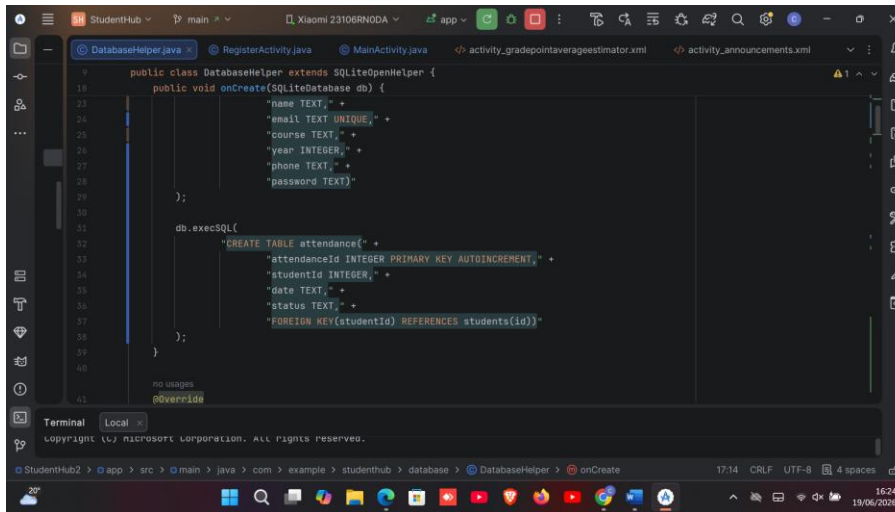
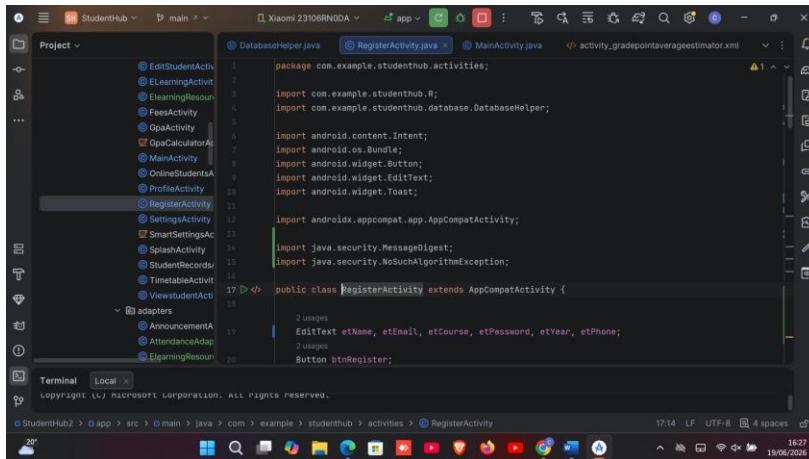


Figure 38.Updated SQLite database structure showing Students and Attendance tables.

Password Hashing Implementation



```
17 public class RegisterActivity extends AppCompatActivity {
21
22     DatabaseHelper databaseHelper;
23
24     @Override
25     protected void onCreate(Bundle savedInstanceState) {
26         super.onCreate(savedInstanceState);
27         setContentView(R.layout.activity_register);
28
29         etName = findViewById(R.id.etName);
30         etEmail = findViewById(R.id.etEmail);
31         etCourse = findViewById(R.id.etCourse);
32         etYear = findViewById(R.id.etYear);
33         etPhone = findViewById(R.id.etPhone);
34         etPassword = findViewById(R.id.etPassword);
35
36         btnRegister = findViewById(R.id.btnRegister);
37
38         databaseHelper = new DatabaseHelper(this);
39
40         btnRegister.setOnClickListener(View v -> {
```

```
41         btnRegister.setOnClickListener(View v -> {
42
43             String name = etName.getText().toString();
44             String email = etEmail.getText().toString();
45             String course = etCourse.getText().toString();
46             String yearStr = etYear.getText().toString();
47             String phone = etPhone.getText().toString();
48             String password = hashPassword(
49                 etPassword.getText().toString()
50             );
51
52             if(name.isEmpty() || email.isEmpty() || course.isEmpty()
53                || yearStr.isEmpty() || phone.isEmpty() || password.isEmpty()) {
54
55                 Toast.makeText(context, RegisterActivity.this,
56                     text: "Fill all fields",
57                     Toast.LENGTH_SHORT).show();
58             } else {
```

```
59             startActivity(new Intent(
60                 packageContext, RegisterActivity.this,
61                 MainActivity.class));
62         } else {
63
64             Toast.makeText(context, RegisterActivity.this,
65                 text: "Registration Failed",
66                 Toast.LENGTH_SHORT).show();
67         }
68     }
69 }
70
71 private String hashPassword(String password) {
```

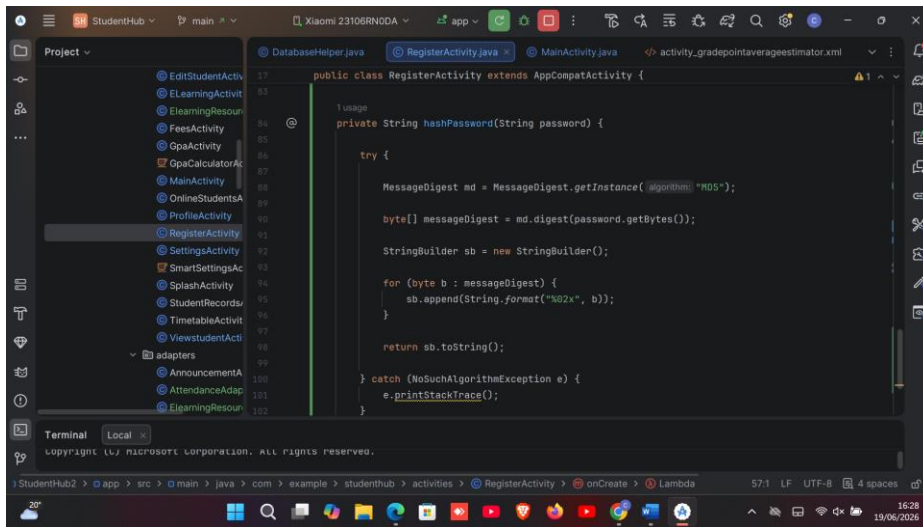
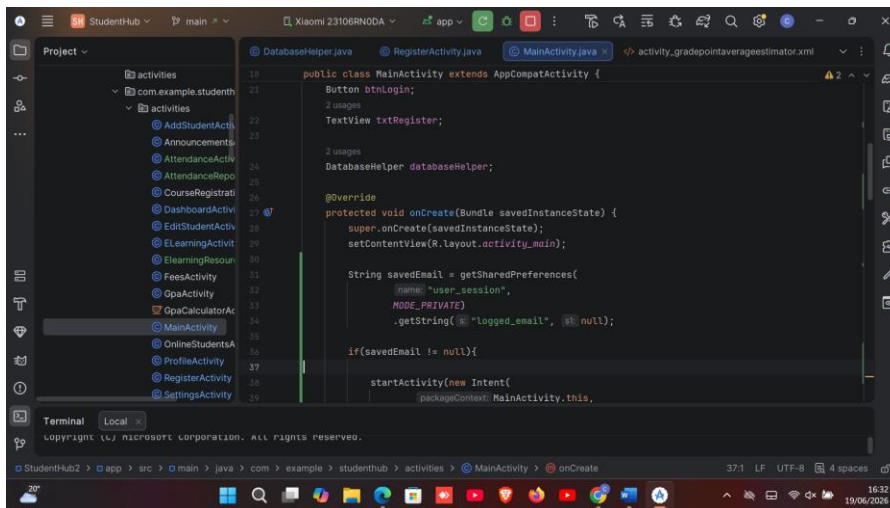
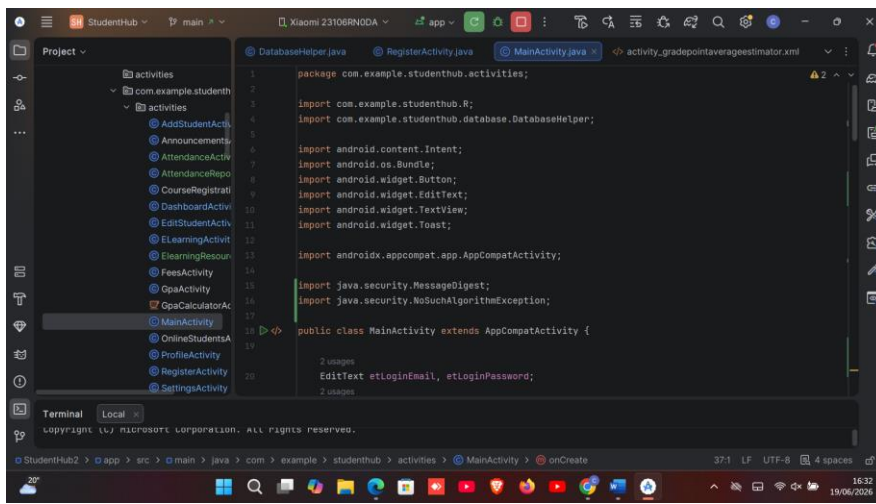


Figure 39 Password hashing implementation for secure user authentication.

Shared Preferences Session



This screenshot shows the MainActivity.java file in an IDE. The MainActivity class extends AppCompatActivity. The onCreate method is protected and takes a Bundle savedInstanceState. It calls finish() and initializes several UI elements: etLoginEmail, etLoginPassword, btnLogin, and txtRegister. It also initializes a DatabaseHelper object. The txtRegister button has a click listener that starts a new activity, RegisterActivity, with the current context.

```
18 public class MainActivity extends AppCompatActivity {
27     protected void onCreate(Bundle savedInstanceState) {
42         finish();
43     }
44
45     etLoginEmail = findViewById(R.id.etLoginEmail);
46     etLoginPassword = findViewById(R.id.etLoginPassword);
47
48     btnLogin = findViewById(R.id.btnLogin);
49     txtRegister = findViewById(R.id.txtRegister);
50
51     databaseHelper = new DatabaseHelper(context);
52
53     txtRegister.setOnClickListener(View v -> {
54
55         startActivity(new Intent(
56             packageContext: MainActivity.this,
57             RegisterActivity.class));
58     });
59
60     btnLogin.setOnClickListener(View v -> {
```

This screenshot shows the MainActivity.java file with the login logic implemented. The btnLogin button has a click listener that gets the password from the etLoginPassword field, hashes it using hashPassword, and then checks the login with databaseHelper.checkLogin. If the login is successful, it creates a session using SharedPreferences, puts the logged email, and shows a success toast. It then starts the RegisterActivity.

```
18 public class MainActivity extends AppCompatActivity {
27     protected void onCreate(Bundle savedInstanceState) {
42         btnLogin.setOnClickListener(View v -> {
43             String password = hashPassword(etLoginPassword.getText().toString());
44
45             boolean checkLogin = databaseHelper.checkLogin(
46                 email, password);
47
48             if(checkLogin) {
49                 // Create Session
50                 getSharedPreferences("user_session", MODE_PRIVATE).
51                     edit().
52                     putString("logged_email", email)
53                     .apply();
54
55                 Toast.makeText(context: MainActivity.this,
56                     text: "Login Successful",
57                     Toast.LENGTH_SHORT).show();
58
59                 startActivity(new Intent(
60                     packageContext: MainActivity.this,
```

This screenshot shows the MainActivity.java file with the hashPassword method implemented. The method takes a password string and returns a hashed password. It uses MessageDigest to create a digest of the password and then appends it to the password string.

```
18 public class MainActivity extends AppCompatActivity {
27     protected void onCreate(Bundle savedInstanceState) {
42         btnLogin.setOnClickListener(View v -> {
43             } else {
44                 Toast.makeText(context: MainActivity.this,
45                     text: "Invalid Email or Password",
46                     Toast.LENGTH_SHORT).show();
47             }
48         });
49     }
50
51     Usage
52     private String hashPassword(String password) {
53         try {
54             MessageDigest md = MessageDigest.getInstance("MD5");
55             byte[] messageDigest = md.digest(password.getBytes());
56             StringBuilder sb = new StringBuilder();
57             for (byte b : messageDigest) {
58                 sb.append(String.format("%02x", b));
59             }
60             return sb.toString();
61         }
```

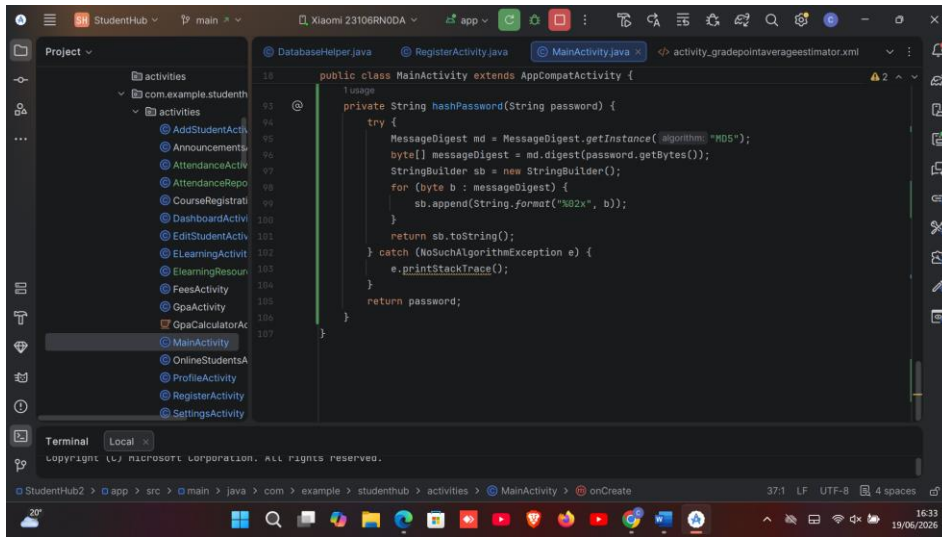



Figure 40 Session management using Shared Preferences.

Attendance Module

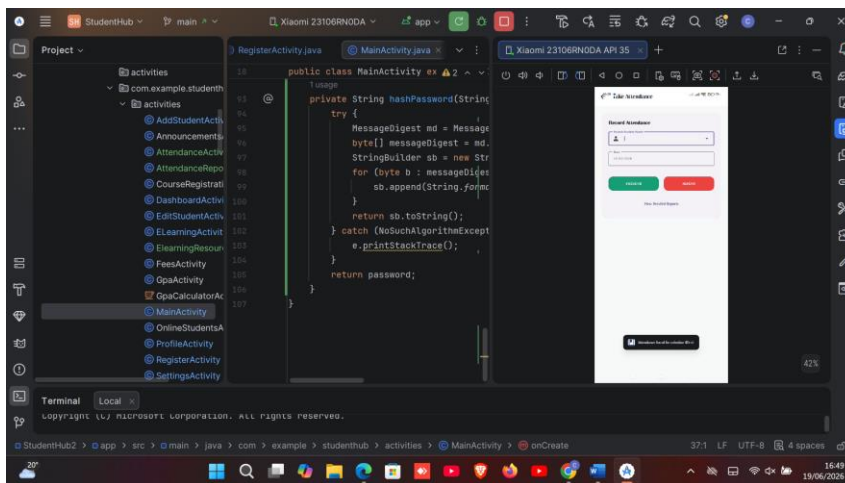


Figure 41. Attendance recording interface

Attendance written & Saved

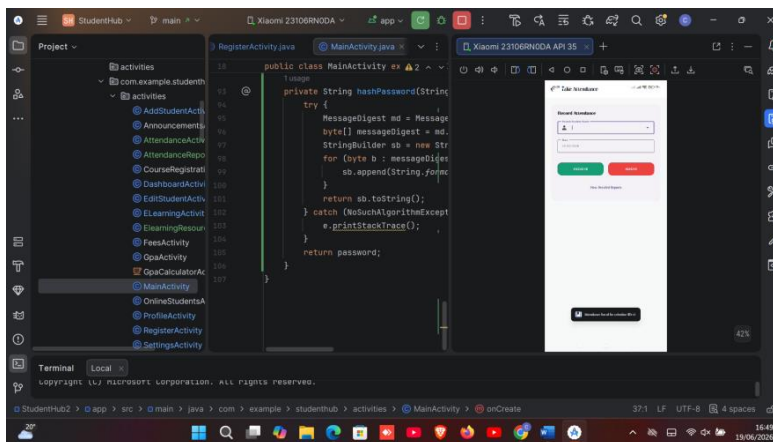
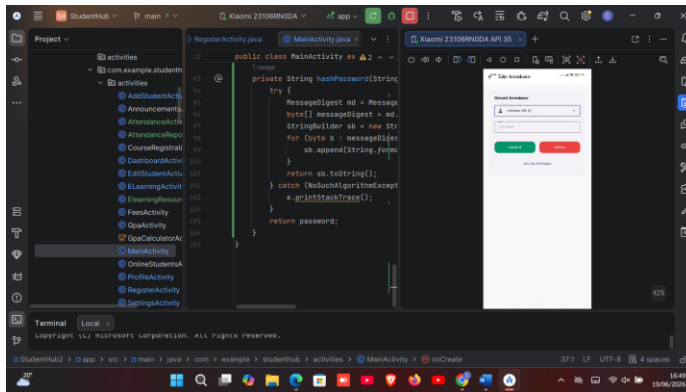


Figure 42Successful attendance record storage

Attendance Reports

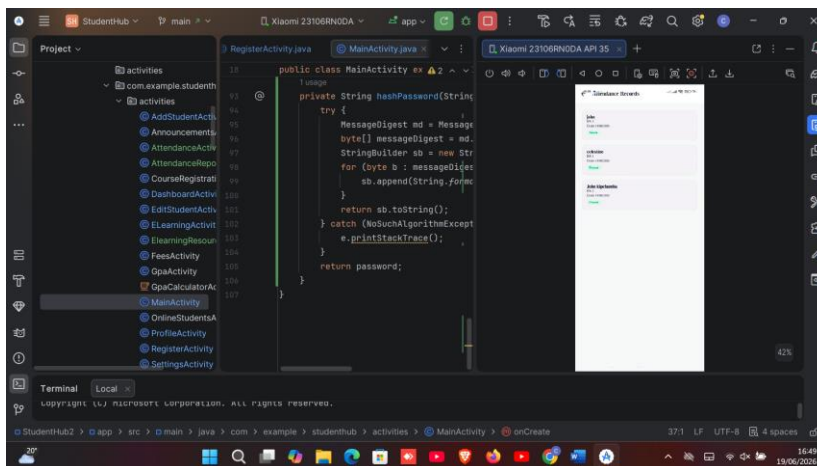


Figure 43Attendance report displaying stored attendance records.

Profile Data Retrieval

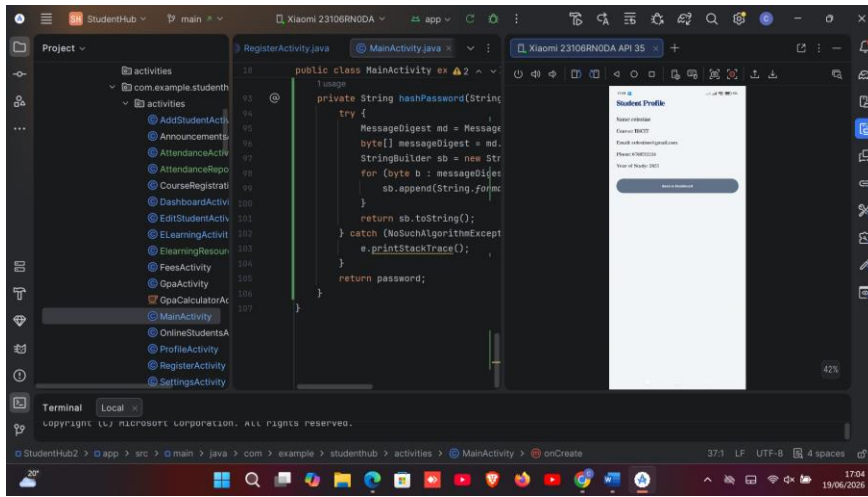


Figure 44 Student profile information retrieved from persistent storage.

TOOLS USED

- Android Studio
- Java
- XML
- SQLite Database
- Shared Preferences
- RecyclerView
- Android Emulator
- Material Design Components

PERSONAL REFLECTION

Week 7 improved my understanding of data storage and database integration. I implemented attendance management, session persistence, password security, and database relationships, enhancing the functionality, security, and reliability of the Student Hub application.

Week 8: Project Planning

PRACTICAL ACTIVITIES

1. Created reusable event-handling classes using Object-Oriented Programming.
2. Implemented keyboard input validation and event handling.
3. Developed touch gesture handling for tap, long press, and swipe events.
4. Added event logging for user interactions.
5. Integrated event handling into the Student Hub application.
6. Tested keyboard and touch gesture functionality.

Events Package Structure

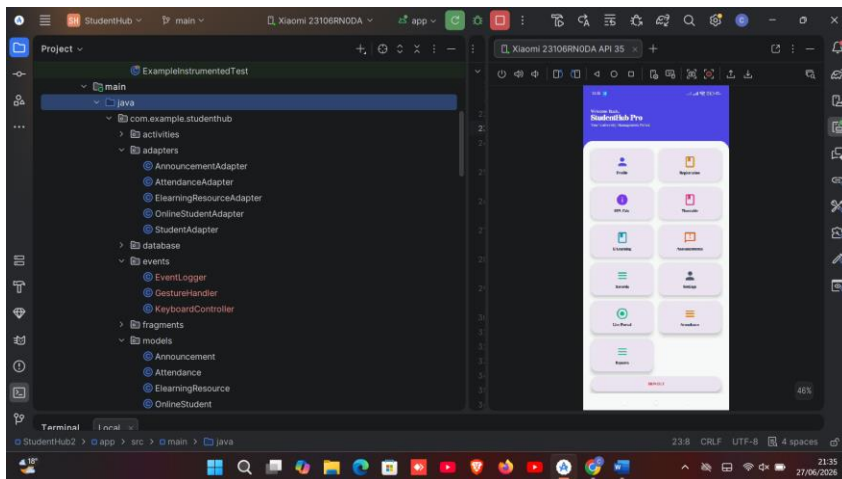


Figure 45Event handling package showing reusable OOP classes.

KeyboardController.java

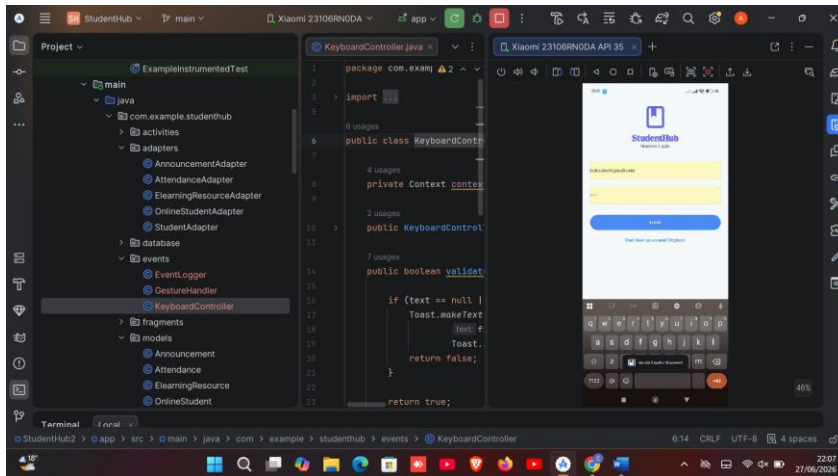


Figure 46 Keyboard input handling and validation class.

GestureHandler.java

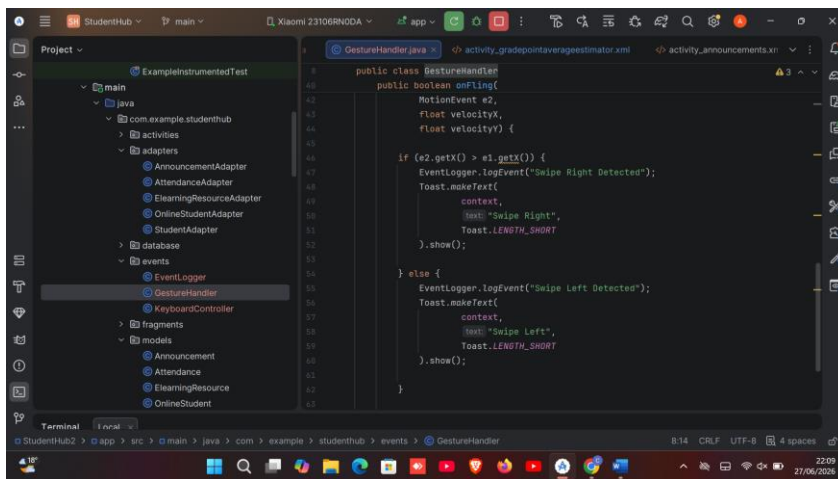
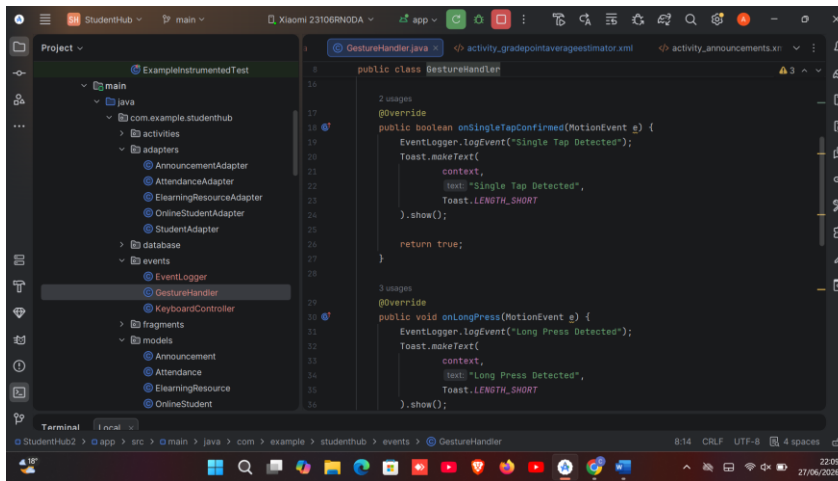


Figure 47 Touch gesture handling implementation.

EventLogger.java

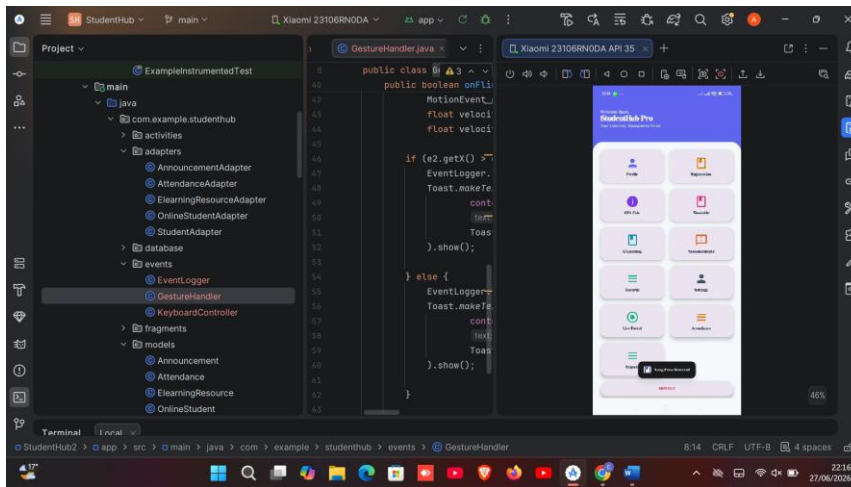
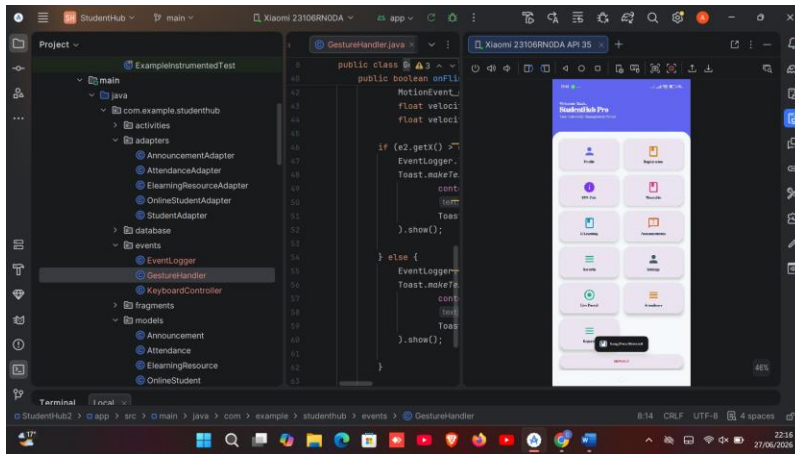


Figure 48 Event logging implementation.

Swipe gesture



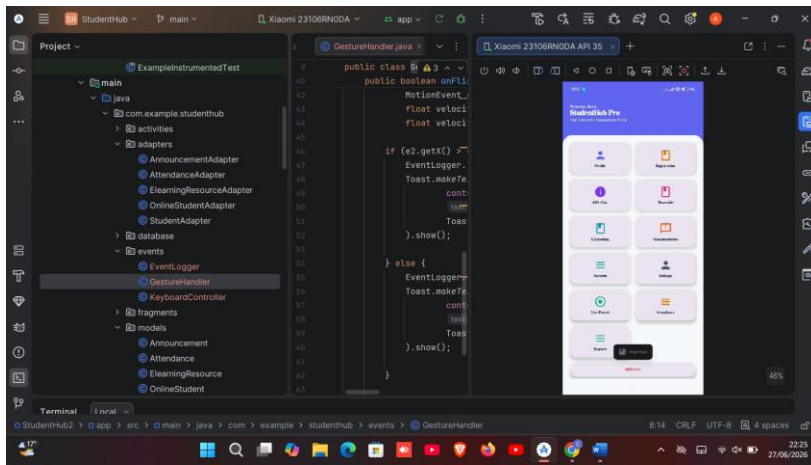
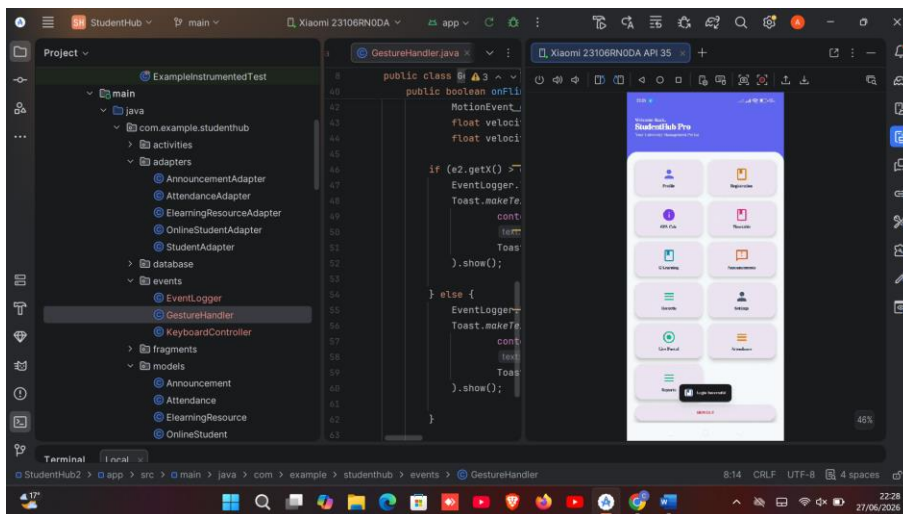


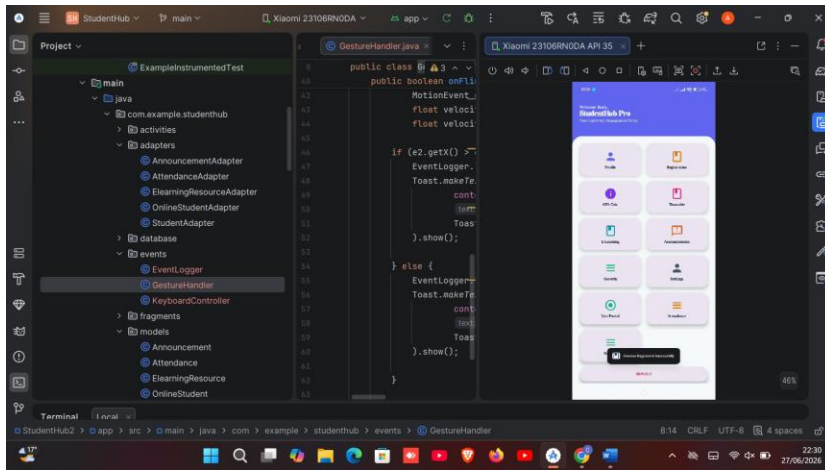
Figure 49swipe gesture from left to right

Event Logger

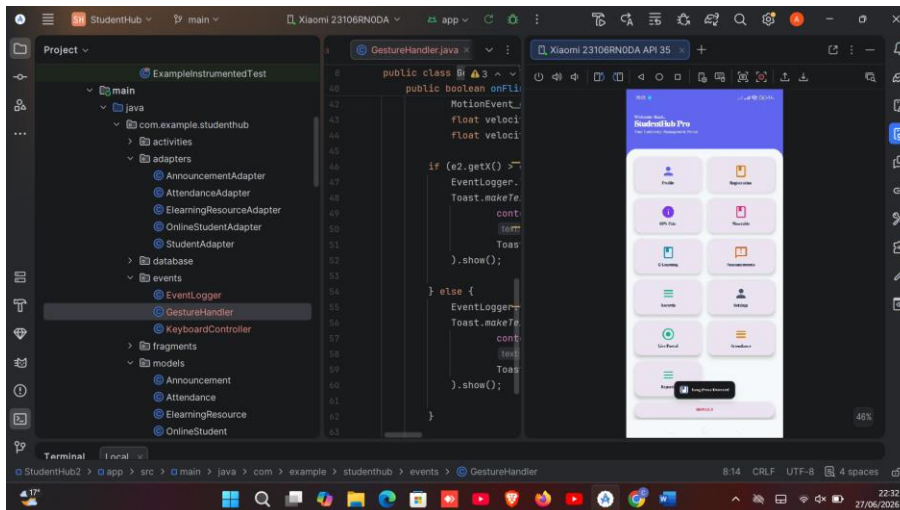
login



Register



longpress



TOOLS USED

- Android Studio
- Java
- XML
- Android SDK
- Gesture Detector API
- Android Emulator
- Logcat

- **Material Design Components**

PERSONAL REFLECTION

Week 8 strengthened my understanding of event-driven programming and object-oriented design. I implemented keyboard input validation, touch gesture handling, and event logging, making the Student Hub application more interactive, modular, and user-friendly.

Week 9: Device Features and Sensors

Practical Activities

- Integrated camera functionality into the application.
- Retrieved user location using GPS.
- Stored campus building information in SQLite.
- Displayed saved building records using RecyclerView.
- Tested runtime permissions for camera and location access.

Campus Explorer Dashboard Card

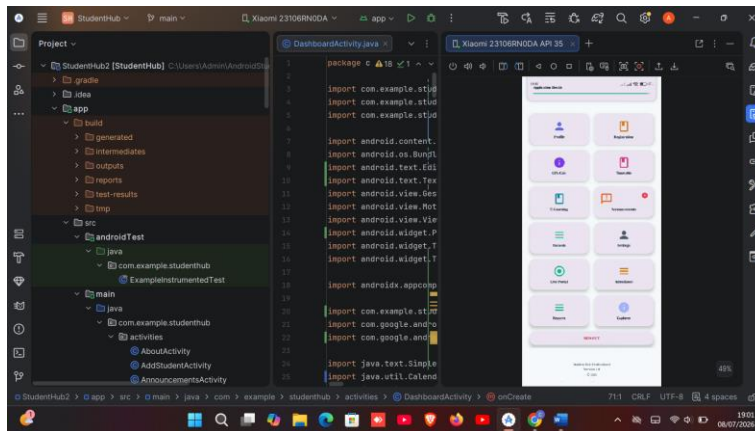


Figure 50new Campus Explorer card.

Campus Explorer Screen

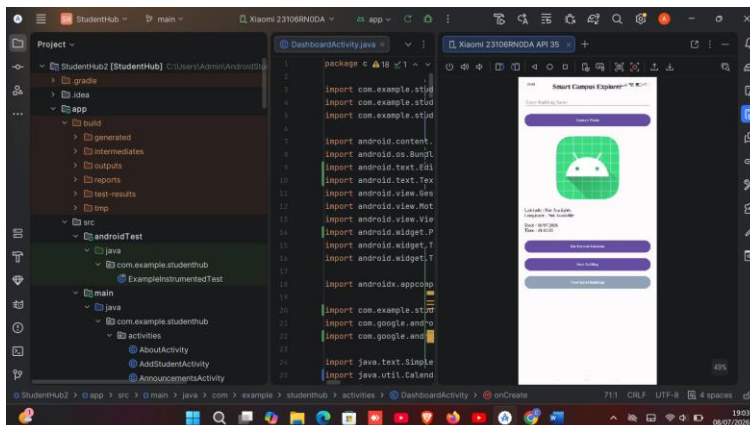


Figure 51 EXPLORER SCREEN

Press Capture Photo AND GPS WORKING

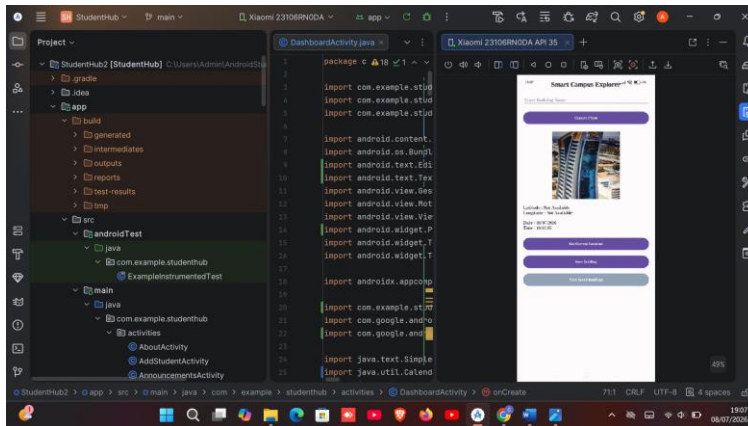


Figure 52 Press Capture Photo AND GPS WORKING

Saved Building

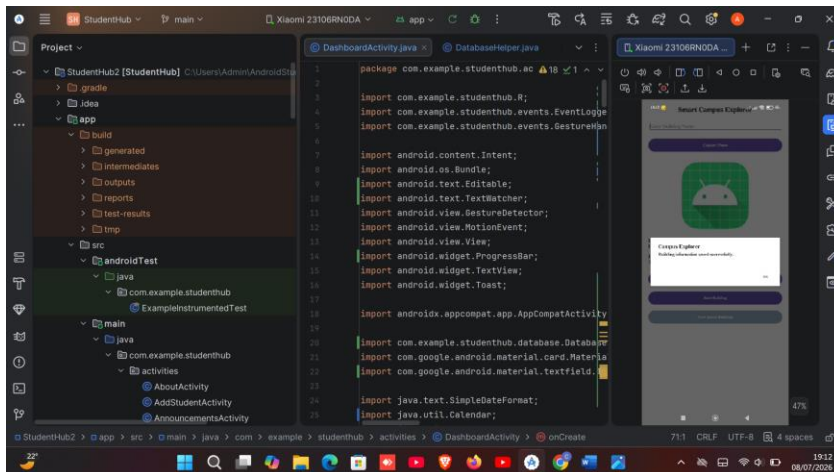


Figure 53 Screenshot showing the success message.

Tools Used

- Android Studio
- Java
- XML
- SQLite Database
- Android Camera
- GPS Location Services
- RecyclerView

- Android Emulator

Personal Reflection

Week 9 enhanced my practical skills in integrating Android device features. I successfully implemented camera and GPS functionality, managed runtime permissions, stored location data in SQLite, and improved my understanding of hardware integration within mobile applications.

Week 10: Notifications and Background Services

Practical Activities

- Integrated all application modules into one system.
- Enhanced the dashboard with professional navigation.
- Improved user interface and user experience.
- Implemented input validation and error handling.
- Tested application functionality and resolved bugs.
- Verified database, camera, GPS, and attendance modules.

Splash Screen

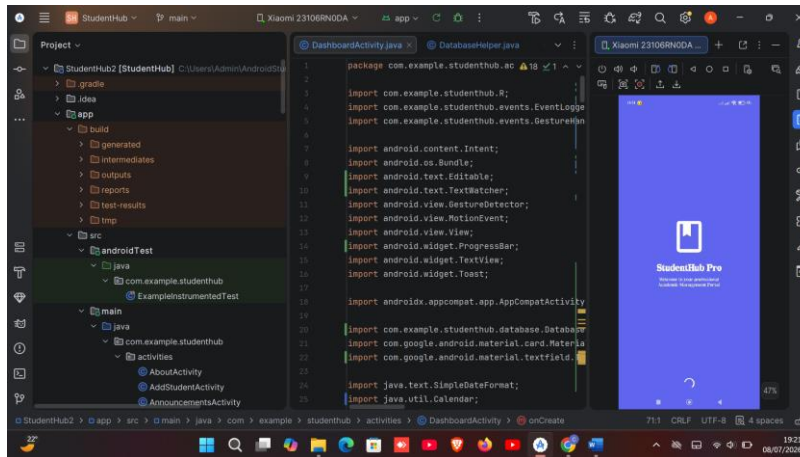
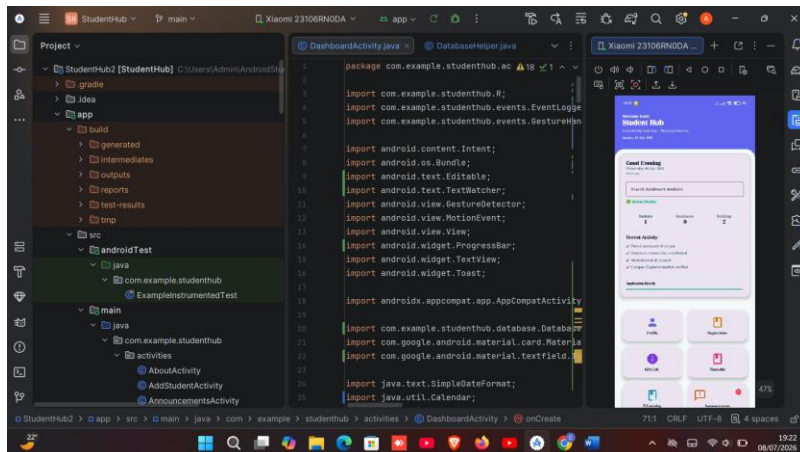


Figure 54 CAPTURED SPLASH SCREEN BEFORE OPENING

Professional Dashboard



Live Dashboard Statistics

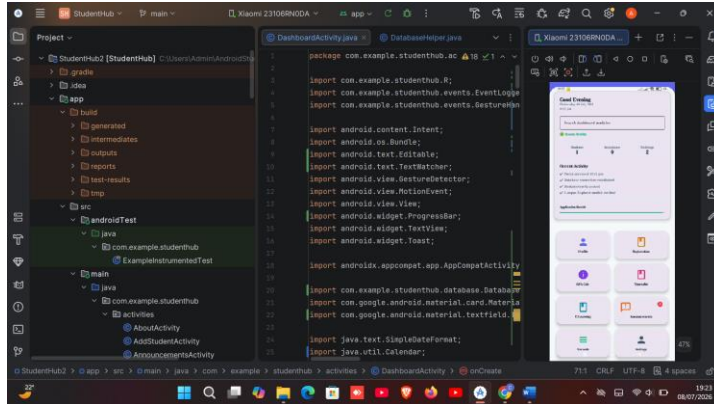
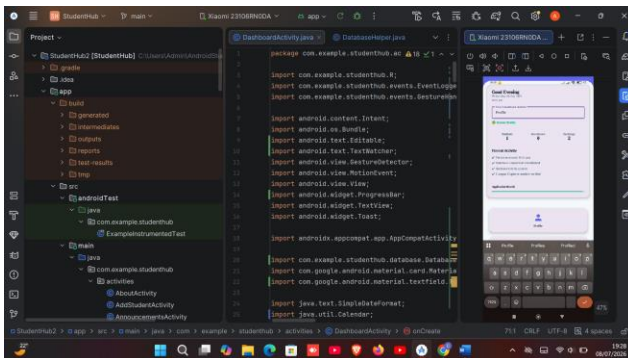


Figure 55Live Dashboard Statistics

Search Function



Tools Used

- Android Studio
- Java
- XML
- SQLite Database
- RecyclerView

- Material Design Components
- Android Emulator
- Logcat

Personal Reflection

Week 10 strengthened my understanding of integrating, testing, and refining a complete Android application. I successfully connected all modules, improved the dashboard, enhanced navigation and validation, fixed bugs through testing, and gained valuable experience in developing a reliable, user-friendly, and professional mobile application.

Week 11: Mobile Security

Practical Activities

- Updated the application for deployment readiness.
- Enhanced the splash screen and launcher icon.
- Added About and Release Notes screens.
- Performed functional testing of all modules.
- Generated a signed APK for release.

Professional Splash Screen

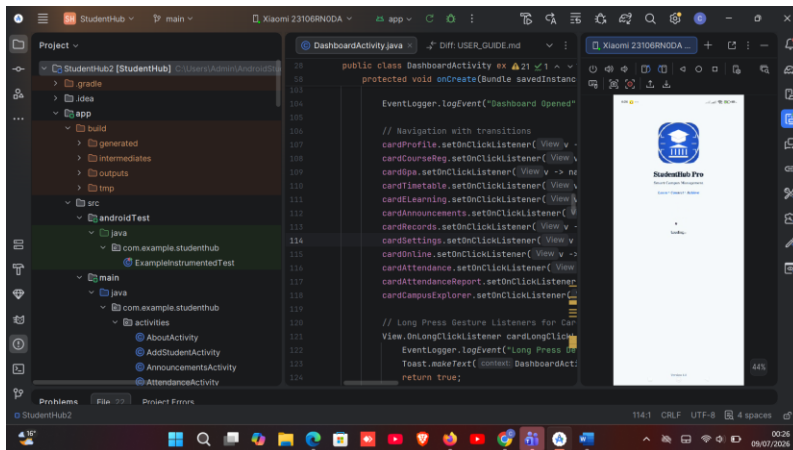


Figure 56 improved Professional Splash Screen with launcher icon

Professional Dashboard

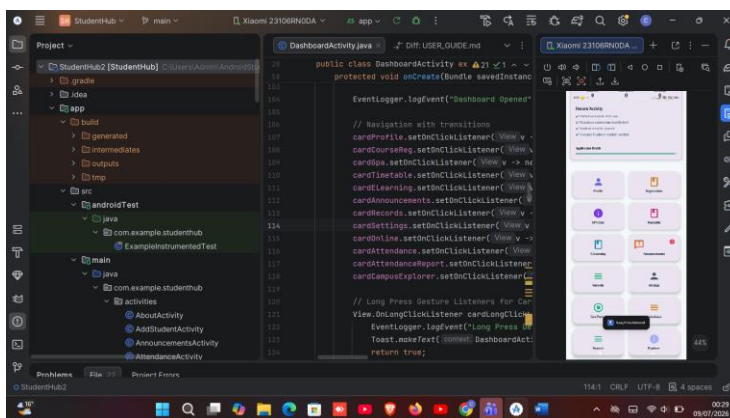


Figure 57all valuable modules in dashboard

About Student Hub

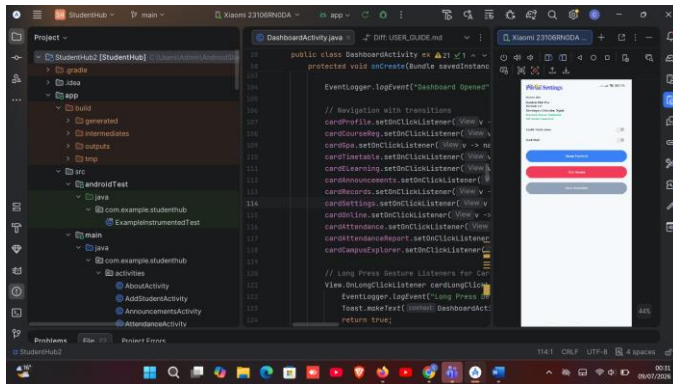


Figure 58 setting about student and release notes on version

Tools Used

- Android Studio
- Java
- XML
- SQLite Database
- Android Emulator
- Material Design Components
- Android SDK
- Gradle Build System

Personal Reflection

Week 11 enhanced my understanding of preparing Android applications for deployment. I improved the application's presentation, verified all modules through testing, generated a signed APK, and documented the project professionally. This strengthened my confidence in delivering a complete, reliable, and deployment-ready mobile application.

Week 12: Testing and Debugging

Same as week 1

Week 13: Application Deployment

Same as week 1

Week 14: Final Project Presentation

Same as week 1

Semester Project Tracking

Project Title

Project Objectives

Project Requirements

Development Milestones

Testing Results

Lessons Learned

Future Improvements

End of Semester Reflection

Summary of Skills Acquired

Most Interesting Topic

Major Challenges Faced

How Challenges Were Overcome

Professional Growth Achieved

Overall Course Experience

Final Approval

Student Name: _____

Student Signature: _____

Date: _____

Lecturer Name: _____

Lecturer Remarks: _____

Date: _____